SIGNSENSE 2.0: ON-DEVICE INDIAN SIGN LANGUAGE RECOGNITION FOR ACCESSIBLE VIDEO CALLING

MACHINE LEARNING – PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF TECHNOLOGY** IN **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** OF THE ANNA UNIVERSITY

MACHINE LEARNING
Nov/Dec 2026



PROJECT WORK

Submitted by

[STUDENT NAME] - [REGISTER NUMBER]

BATCH
2024 – 2028

Under the Guidance of

N. POORANAM, M.E., (Ph.D)
ASSISTANT PROFESSOR
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



Sri Eshwar College of Engineering
(An Autonomous Institution)
Kinathukadavu (Tk), Coimbatore - 641 202, Tamil Nadu
Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai



DECLARATION

I declare that the work entitled **"SignSense 2.0: On-Device Indian Sign Language Recognition for Accessible Video Calling"** is submitted in partial fulfillment of the requirement for the award of the degree in B. Tech. Artificial Intelligence and Data Science, Anna University Chennai, is a record of my own work carried out by me during the academic year 2025-2026 under the supervision and guidance of **Ms.N.Pooranam**, Assistant Professor, Department of Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places. The matter embodied in this work is original and has not been submitted for the award of any other degree or diploma, either in this or any other University.

Student Name	**Register Number**	**Signature**
[STUDENT NAME]	[REGISTER NUMBER]	

I certify that the declaration made above by the candidate is true.

Ms.N.Pooranam
Assistant Professor
Department of AI&DS

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

Certified that this Report titled **"SignSense 2.0: On-Device Indian Sign Language Recognition for Accessible Video Calling"** is the bonafide work of

[STUDENT NAME] **[REGISTER NUMBER]**

who carried out the project work under my supervision.

SIGNATURE

Dr.G. Sathish Kumar M.E.,PhD.,
HEAD OF THE DEPARTMENT
Artificial Intelligence and Data Science,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

SIGNATURE

N.Pooranam,M.E.,(Ph.D)
SUPERVISOR
Assistant Professor,
Artificial Intelligence and Data Science,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the **Autonomous Semester End Project on Machine Learning Viva-Voce** held on

……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I thank the Almighty for giving me an opportunity to study in a prestigious institution and grace to complete this project successfully.

I also express my gratitude to our Chairperson **Mr. R. Mohanram** and Director **Mr. R. Rajaram,** Sri Eshwar College of Engineering for their endeavor to provide all the facilities we require and the interest they showed in the welfare of the students.

I wish to express my sincere thanks to **Dr. Sudha Mohanram,** the **Principal & Joint Managing Trustee** of our college for providing the necessary infrastructure facilities.

I express my sincere gratitude to **Dr. G. Sathish Kumar** the **Head of the Department,** Artificial Intelligence and Data Science**,** for his guidance and for providing valuable suggestions and all necessary facilities at the right time for doing this project.

I express my gratitude and sincere thanks to my guide **Ms. N. Pooranam, Assistant Professor,** Dept. of Artificial Intelligence and Data Science, for her valuable suggestions and constant encouragement for successful completion of this project.

I would also extend my hearty thanks to my family members and all the friends who encouraged me to do this project successfully.

QUALITY POLICY

To establish a system of Quality Enhancement, which would on a continuous basis evaluate and enhance the quality of teaching – learning, research and extension activities of the institution, leading to improvements in all processes, enabling the institution to attain excellence.

INSTITUTE VISION

To be recognized as a premier institution, grooming students into globally acknowledged engineering professionals.

INSTITUTE MISSION

- Providing outcome and value-based engineering education
- Nurturing research and entrepreneurial culture
- Enabling students to be industry ready and fulfill their career aspirations
- Grooming students through behavioral and leadership training programs
- Making students socially responsible

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT VISION

To build a conducive academic and research environment in the stream of Artificial Intelligence and Data Science for enabling global education, research and entrepreneurship.

DEPARTMENT MISSION

- Update and enrich the curriculum by integrating global trends and technological advancements to ensure the academic excellence.
- Promote a research-driven environment by providing access to state-of-the-art infrastructure with emerging technologies.
- Empower the faculty through continuous upskilling, international collaborations, and active research engagement to provide globally competent education.
- Encourage student-centric, industry-aligned learning that nurtures innovation and entrepreneurship activities for solving real-world and societal challenges.

PROGRAM EDUCATIONAL OBJECTIVES

PEO1: Build a progressive professional career in industry, or pursue higher education, or succeed as an entrepreneur.

PEO2: Excell in cutting-edge technologies of Artificial Intelligence and Data Science to address real world challenges.

PEO3: Exhibit professional ethics, involvement in team work in their profession and contributing to the advancement of society.

PROGRAM OUTCOMES

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

PROGRAM SPECIFIC OUTCOMES

PSO1. Apply programming, problem-solving, and hardware integration skills with AI and data science to design intelligent, secure and efficient systems.

PSO2. Develop an AI-driven application by exploring optimized coding competencies using predictive modeling and data analytics technologies.

ABSTRACT

Deaf people who use Indian Sign Language (ISL) often cannot talk easily with family members who do not sign, and many of them are not comfortable reading long text. Ordinary video-calling tools carry the picture and the sound, but they do not help the two people understand each other. SignSense 2.0 is a web-based video-calling platform that adds sign-language assistance in both directions: supported ISL signs made in front of the camera become text and speech for the hearing person, and what the hearing person types or says is signed back to the signer by Saathi, an animated ISL signer, alongside a one-tap picture board.

The recognition pipeline runs entirely in the signer's browser. MediaPipe landmark models, executed in a Web Worker, track the hands, upper body and face; a rule-based segmenter cuts each sign from hands-up to hands-down; a 333-dimensional, body-normalised feature sequence is classified by a compact temporal convolutional network (SignCNN, 876,329 parameters) exported to ONNX. A calibrated decision policy, trained with real out-of-vocabulary ISL signs as an 'unknown' class, decides whether a sign is sent automatically, offered to the signer as a choice, or rejected, so that uncertain or unsupported gestures do not silently become words.

The system is built with Next.js, TypeScript, FastAPI, PostgreSQL, WebRTC, PyTorch and ONNX Runtime Web, using the INCLUDE dataset (39 everyday signs, 708 clips, plus 669 clips of 223 other words as negatives). On the official INCLUDE test split the published model reached 96.7% top-1 accuracy with no wrong word sent automatically, and it reduced unknown signs sent as words from 55.3% to 5.0% compared with the earlier model; on held-out recording sessions it reached 86.4% top-1 and 94.5% top-3 accuracy. All automated suites pass, including a two-browser end-to-end call test that feeds recorded test clips through the real camera path. Tests with real people at physical webcams and Deaf user evaluation remain future work.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	LIST OF TABLES	**XII**
	LIST OF FIGURES	**XIII**
	LIST OF ABBREVIATIONS	**XIV**
1.	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 MOTIVATION	1
	1.3 OBJECTIVES	2
	1.4 SCOPE OF THE PROJECT	2
	1.5 ORGANIZATION OF THE REPORT	2
2.	**LITERATURE REVIEW**	**3**
	2.1 SIGN LANGUAGE RECOGNITION	3
	2.2 INDIAN SIGN LANGUAGE DATASETS	3
	2.3 LANDMARK ESTIMATION	3
	2.4 TEMPORAL MODELS FOR SEQUENCES	4
	2.5 CALIBRATION AND REJECTION OF UNKNOWN INPUTS	4
	2.6 REAL-TIME COMMUNICATION ON THE WEB	4
	2.7 SUMMARY AND RESEARCH GAP	5
3.	**PROBLEM DEFINITION**	**6**

	3.1 EXISTING SYSTEMS	6
	3.2 DISADVANTAGES OF EXISTING SYSTEM	6
	3.3 PROPOSED SYSTEM	6
	3.4 ADVANTAGES OF PROPOSED SYSTEM	7
4.	**SYSTEM STUDY**	**8**
	4.1 FEASIBILITY STUDY	8
	4.1.1 Economical Feasibility	8
	4.1.2 Technical Feasibility	8
	4.1.3 Operational Feasibility	8
	4.2 USER REQUIREMENTS	8
	4.2.1 Functional Requirements	8
	4.2.2 Non-Functional Requirements	9
5.	**SYSTEM REQUIREMENTS SPECIFICATION**	**10**
	5.1 SOFTWARE REQUIREMENTS	10
	5.2 HARDWARE REQUIREMENTS	10
	5.3 SOFTWARE DESCRIPTION	10
	5.4 DATASET DESCRIPTION	11
6.	**SYSTEM DESIGN**	**12**
	6.1 SYSTEM ARCHITECTURE	12
	6.2 DATA FLOW DIAGRAM	13
	6.3 LIVE RECOGNITION PIPELINE	14
	6.4 DECISION POLICY	14
	6.5 DATABASE DESIGN	15
	6.6 CALL SIGNALLING SEQUENCE	16
	6.7 USER INTERFACE DESIGN	17
7.	**SYSTEM IMPLEMENTATION**	**18**
	7.1 MODULES	18
	7.2 MODULE DESCRIPTION	18
	7.2.1 Authentication, Preferences and Meetings	18
	7.2.2 Video Calling and Signalling	18

	7.2.3 Landmark Extraction	19
	7.2.4 Sign Segmentation and Feature Engineering	19
	7.2.5 Sign Classification Model and Training	19
	7.2.6 Calibration and Decision Policy	20
	7.2.7 Text-to-Sign Generation, Picture Board and Captions	20
	7.2.8 Automated ML Pipeline and Evaluation	21
	7.2.9 Algorithm	21
8.	**SYSTEM TESTING AND MAINTENANCE**	**22**
	8.1 UNIT TESTING	22
	8.2 INTEGRATION TESTING	22
	8.3 FUNCTIONAL TESTING	23
	8.4 SYSTEM TESTING	23
	8.4.1 Model Evaluation	23
	8.4.2 Ablation Study	26
	8.4.3 Rejection Study	27
	8.4.4 End-to-End Call Testing	27
	8.4.5 Performance	28
	8.5 WHITE BOX TESTING	29
	8.6 BLACK BOX TESTING	29
	8.7 SECURITY TESTING	29
9.	**FUTURE ENHANCEMENT AND CONCLUSION**	**30**
	APPENDIX	**31**
	A.1 SOURCE CODE	31
	A.2 SCREENSHOTS	34
	REFERENCES	**39**

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1	Summary of Related Work	5
2	Dataset Summary	11
3	Backend API Endpoints	16
4	Training Configuration	20
5	Representative Unit Test Cases and Outcomes	22
6	Functional Test Journeys	23
7	Comparison of the Earlier and Published Models (Official Test Split)	24
8	Official versus Session-Held-Out Evaluation	25
9	Ablation Results (Mean of Three Seeds)	26
10	Rejection Rules Evaluated (Official Test Split)	27
11	End-to-End Call Tests	28
12	Performance Measurements	28

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
6.1	System Architecture	12
6.2	Data Flow Diagram	13
6.3	Live Recognition Pipeline	14
6.4	Decision Policy	15
6.5	Entity Relationship Diagram	15
6.6	Call Signalling Sequence	17
7.1	SignCNN Model Architecture	20
8.1	Earlier versus Published Model	24
8.2	Official versus Session-Held-Out Results	25
8.3	Confusion Matrix of the Published Model	26
8.4	Practice Page Before and After the Web Worker	29
A.2.1	Landing Page	34
A.2.2	Dashboard	34
A.2.3	Pre-Join Screen	35
A.2.4	Call Screen	35
A.2.5	Live Sign Recognition in a Call	36
A.2.6	Picture Board on a Phone	37
A.2.7	ISL Sign Dictionary	38
A.2.8	Practice Page	38

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASL	American Sign Language
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
FAR	False Acceptance Rate
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
ICE	Interactive Connectivity Establishment
ISL	Indian Sign Language
JWT	JSON Web Token
ML	Machine Learning
ONNX	Open Neural Network Exchange
OOV	Out-Of-Vocabulary
SDP	Session Description Protocol
STUN	Session Traversal Utilities for NAT
TTS	Text-To-Speech
TURN	Traversal Using Relays around NAT
UI	User Interface
WebRTC	Web Real-Time Communication

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Sign languages are complete natural languages with their own grammar, vocabulary and regional variation. Indian Sign Language (ISL) is used by a large Deaf community across India, yet most hearing family members, doctors and service providers never learn it. Communication inside a family therefore often falls back on improvised gestures, lip-reading and short written notes. Written notes are a weak bridge: many Deaf adults learned the written language as a second language and find long text tiring or confusing, and elderly relatives may not read comfortably either.

Video calling has become an everyday way for families to stay in touch, and it is especially valuable for signers because it carries the visual channel that sign language needs. However, common video-calling tools only transport the picture and the sound. When one person signs and the other does not, the call shows the signs clearly but does nothing to help the hearing person understand them, and nothing to help the signer follow speech beyond optional captions.

Recent progress in on-device machine learning makes a different approach possible. Lightweight landmark models such as Google MediaPipe can track the hands, body and face in real time inside an ordinary web browser, and small neural networks exported to ONNX can classify short landmark sequences in a few milliseconds. This means that a modest vocabulary of signs can be recognised on the signer's own device, without uploading video to a server.

1.2 MOTIVATION

The project was motivated by a simple scenario: a Deaf person who may not read well, talking with family members who do not sign. For this pair, a useful tool must be visual first. The signer should be able to sign and have the other person see and hear the meaning; the hearing person should be able to speak or type and have the signer see signs and pictures rather than paragraphs of text. Equally important, the tool must not put words in the signer's mouth: an incorrect automatic translation in a family or medical conversation can be worse than no translation at all.

SignSense 2.0 is designed around these two principles: communication in both directions, and honest handling of uncertainty. When the recogniser is sure, a sign is sent after a short countdown that the signer can cancel; when it is unsure, the signer chooses from a few options; and when the movement is not a supported sign, nothing is sent.

1.3 OBJECTIVES

1. **Two-way communication:** Build a one-to-one video call in which supported ISL signs become text and speech, and typed or spoken words are signed in ISL by an animated signer.
2. **On-device recognition:** Run landmark extraction and sign classification in the browser so that video never leaves the signer's device.
3. **Safe decisions:** Calibrate the model's confidence and add an 'unknown' class so that uncertain or unsupported gestures are not sent as words.
4. **Visual-first design:** Provide animated signs, a picture board and large controls so that the core flow does not require reading.
5. **Measured evaluation:** Evaluate the model on held-out test data, report its failure modes honestly, and verify the full call workflow with automated tests.

1.4 SCOPE OF THE PROJECT

The current release supports one-to-one calls and isolated (word-level) recognition of 39 everyday ISL signs from the INCLUDE dataset, covering greetings, family members, health, feelings and time. Every supported sign is also performed by Saathi, a drawn signer, for text-to-sign playback. Continuous signing, full ISL sentence grammar, fingerspelling and group calls are outside the scope of this version. The platform runs in Chrome or Edge on laptops and phones; recognition needs a webcam, while guests can join with only a link.

1.5 ORGANIZATION OF THE REPORT

Chapter 2 reviews related work on sign language recognition, landmark estimation, confidence calibration and real-time communication. Chapter 3 defines the problem and the proposed system, Chapter 4 studies its feasibility, and Chapter 5 lists the requirements and the dataset. Chapter 6 presents the system design, Chapter 7 describes the implementation of each module, Chapter 8 reports the testing and evaluation results, and Chapter 9 discusses future enhancements and concludes the report. The appendix contains selected source code and screenshots of the application.

CHAPTER 2

LITERATURE REVIEW

2.1 SIGN LANGUAGE RECOGNITION

Automatic sign language recognition has been studied for several decades. Early systems relied on data gloves or coloured markers, which are accurate but intrusive. Vision-based systems later learned directly from video, first with hand-crafted features and hidden Markov models and then with deep networks. Koller (2020) surveys this progress and notes that most published results are on a small number of benchmark datasets and are rarely evaluated on unseen signers, which limits how well reported accuracies transfer to real use.

Recognition is usually divided into isolated recognition, where each clip contains one sign, and continuous recognition or translation, where a sentence must be segmented and translated. Camgoz et al. (2018) framed continuous sign language translation as a neural machine translation problem. Large word-level datasets such as WLASL for American Sign Language (Li et al., 2020) made it possible to compare pose-based and appearance-based models at scale. These works show that word-level recognition of a limited vocabulary is achievable, whereas open-vocabulary translation remains an open research problem.

2.2 INDIAN SIGN LANGUAGE DATASETS

ISL has far fewer public resources than ASL. Sridhar et al. (2020) released INCLUDE, a dataset of 4,287 videos covering 263 everyday ISL words in 15 categories, recorded by Deaf students of St. Louis School for the Deaf, Chennai, and published under the CC BY 4.0 licence with an official train/test split. INCLUDE is the main data source for this project: 39 of its words form the supported vocabulary and the remaining 223 words provide realistic examples of signs that the system must reject. The dataset does not publish signer identities, so its official split cannot be verified to be signer-independent.

2.3 LANDMARK ESTIMATION

Instead of classifying raw pixels, many recent systems first estimate body landmarks. MediaPipe (Lugaresi et al., 2019) is a framework for on-device perception pipelines; its hand model predicts 21 three-dimensional keypoints per hand in real time (Zhang et al., 2020), BlazePose estimates 33 body keypoints (Bazarevsky et al., 2020), and the face mesh model tracks facial geometry (Kartynnik et al., 2019). Landmarks are compact, largely independent of clothing and background, and cheap to classify, which makes them well suited to recognition inside a web browser.

2.4 TEMPORAL MODELS FOR SEQUENCES

A sign is a movement, so the classifier must model time. Recurrent networks such as the Gated Recurrent Unit (Cho et al., 2014) process a sequence step by step. Bai et al. (2018) showed that temporal convolutional networks with dilated convolutions match or outperform recurrent networks on many sequence tasks while being simpler and faster to run. This project compares a bidirectional GRU with a dilated temporal CNN and selects the CNN because both reach the same accuracy and the CNN is lighter in the browser.

2.5 CALIBRATION AND REJECTION OF UNKNOWN INPUTS

Neural networks tend to be over-confident. Guo et al. (2017) showed that modern networks are poorly calibrated and that a single temperature parameter fitted on validation data greatly improves calibration. Hendrycks and Gimpel (2017) proposed the maximum softmax probability as a baseline for detecting misclassified and out-of-distribution inputs, and Liu et al. (2020) proposed energy scores for the same purpose. These ideas are central to SignSense: a family conversation must not receive a confident but wrong word, so the system calibrates its probabilities, trains an explicit 'unknown' class from real unsupported signs, and evaluates energy- and margin-based rejection rules.

2.6 REAL-TIME COMMUNICATION ON THE WEB

WebRTC (W3C, 2021) lets browsers exchange audio, video and arbitrary data directly, using a signalling channel only to exchange session descriptions and network candidates. STUN (Rosenberg et al., 2008) lets a browser discover its public address, and TURN (Mahy et al., 2010) relays traffic when a direct connection is impossible. Because media flows peer-to-peer, the server never sees the video, which matches the privacy goals of this project.

2.7 SUMMARY AND RESEARCH GAP

Table 1 Summary of Related Work

Work	Approach	Data	Relevance / Gap
Sridhar et al. (2020)	Pose and appearance models for isolated ISL words	INCLUDE (263 ISL words)	Main ISL dataset; signer identities not released
Li et al. (2020)	Pose-based and video-based word-level recognition	WLASL (ASL)	Shows scale benefits; ASL, not ISL
Camgoz et al. (2018)	Neural sign language translation	German Sign Language weather data	Continuous translation; heavy models
Zhang et al. (2020)	Real-time on-device hand landmarks	MediaPipe Hands	Enables in-browser tracking
Bai et al. (2018)	Dilated temporal convolutions vs recurrent nets	Sequence benchmarks	Basis for the SignCNN design
Guo et al. (2017)	Temperature scaling for calibration	Image classification	Used to calibrate sign confidence
Hendrycks & Gimpel (2017)	Softmax-based out-of-distribution detection	Vision and text	Motivates unknown-sign rejection

Existing work focuses on recognition accuracy on benchmark data and on translation research systems. Very little of it is packaged as an accessible, privacy-preserving communication tool for ISL users, and almost none explicitly measures what a system does with signs it was not trained on. SignSense addresses this gap by combining an in-browser recogniser with a calibrated accept / suggest / reject policy inside a working video call, and by reporting both accuracy and false acceptance of unknown signs.

CHAPTER 3

PROBLEM DEFINITION

3.1 EXISTING SYSTEMS

Families with a Deaf member currently rely on general-purpose tools. Standard video-calling applications carry video and audio and sometimes offer automatic captions of speech, but they offer no support for sign language. Human ISL interpreters provide the best quality, but they are scarce, expensive and impractical for everyday family conversations. Text messaging works only for people who read and write comfortably. Research prototypes for sign recognition exist, but they are usually offline demonstrations that classify pre-recorded clips, require powerful hardware, or send video to a server for processing.

3.2 DISADVANTAGES OF EXISTING SYSTEM

1. **One-Way Help Only:** Captions help a reader follow speech, but nothing turns signs into words for the hearing person.
2. **Reading Required:** Text chat and captions assume that both people read comfortably.
3. **Privacy Concerns:** Cloud-based recognition requires uploading continuous video of people's homes and faces.
4. **Over-Confident Output:** Classifiers always output their best guess, even for gestures they have never seen.
5. **Limited ISL Support:** Most tools and datasets target ASL; ISL resources are scarce.
6. **Not Integrated:** Recognition demos are separate from the calls in which communication actually happens.

3.3 PROPOSED SYSTEM

SignSense 2.0 is a web application for one-to-one video calls with ISL assistance in both directions. During a call, the signer's browser extracts hand, body and face landmarks in a background worker, detects the start and end of each sign, and classifies the sign with a small neural network. A calibrated decision policy then either sends the word after a cancellable two-second countdown, asks the signer to choose among the three most likely signs, or reports that no supported sign was detected. Accepted words travel over the WebRTC data channel and appear for the hearing person as large captions, in the transcript and as speech. In the other direction, typed or spoken words are signed by Saathi, a drawn ISL signer, and a picture board offers one-tap messages such as water, pain or help. Accounts, meetings and signalling are

handled by a FastAPI backend with PostgreSQL, while audio, video and messages flow directly between the browsers.

3.4 ADVANTAGES OF PROPOSED SYSTEM

1. **Two-Way Communication:** Signs become text and speech, and words become animated signs, inside the same call.
2. **On-Device Privacy:** Video and landmarks never leave the signer's browser; the server only relays connection set-up messages.
3. **Safe Decisions:** Calibrated confidence, an unknown-sign class and agreement across the clip prevent confident wrong words.
4. **Signer in Control:** Every automatic send can be cancelled, and uncertain results require an explicit choice.
5. **Visual-First Interface:** Animated signs, pictures and large controls make the core flow usable without reading.
6. **No Installation:** Runs in a standard browser; guests join with a link and need no account.
7. **Low Cost:** Built entirely from open-source software and runs on ordinary laptops and phones.
8. **Measured Behaviour:** Accuracy, rejection of unknown signs and call reliability are measured by automated tests.

CHAPTER 4

SYSTEM STUDY

4.1 FEASIBILITY STUDY

A feasibility study assesses whether SignSense can be built and used with the available resources. The analysis covers economic, technical and operational feasibility.

4.1.1 Economical Feasibility

All components are open source: Next.js, React and Tailwind for the interface, FastAPI and PostgreSQL for the backend, PyTorch for training, ONNX Runtime Web and MediaPipe for in-browser inference. The INCLUDE dataset is freely available under CC BY 4.0. Training was carried out on the developer's own laptop with an NVIDIA RTX 4060 GPU, and no paid cloud service or API is used. Because recognition runs on users' devices, the server needs no GPU and hosting cost stays low; the only optional running cost for a public deployment is a TURN relay server for calls across restrictive networks.

4.1.2 Technical Feasibility

The published model has 876,329 parameters and a 3.5 MB ONNX file. One decision, which classifies three window placements of a sign, takes about 10 ms on a single CPU thread. Landmark extraction takes about 35 ms per frame in a Web Worker on the development laptop, which is enough for the 15 frames per second that the model uses. WebRTC, Web Workers, WebAssembly and WebGL are supported by current Chrome and Edge browsers on desktops and phones, so no plug-in or native application is needed.

4.1.3 Operational Feasibility

The application is operated entirely through a web browser. A user creates an account, presses New meeting and sends the link; the other person opens the link and joins as a guest. Recognition starts automatically for signers and shows its state in plain words. Pictures, animated signs and large buttons keep the core flow usable for people who do not read well. Clear error messages explain camera permission problems and connection loss, and the call recovers automatically after short network interruptions.

4.2 USER REQUIREMENTS

The requirements were derived from the target scenario of a Deaf signer talking with hearing family members.

4.2.1 Functional Requirements

1. Users can sign up, sign in, set their communication preferences and delete their account.

2. A signed-in user can create a meeting and share a link; anyone with the link can join as a guest.
3. The call provides two-way video and audio, camera and microphone controls, and a host-only option to end the meeting.
4. Supported ISL signs are recognised during the call and delivered as text and speech after the signer's confirmation rules.
5. Typed and spoken words are signed to the signer by Saathi where a sign exists, and shown as text otherwise.
6. A picture board sends common needs with one tap, and a transcript records the conversation for both sides.
7. Signs can be practised outside a call, and every supported sign can be watched in a dictionary.

4.2.2 Non-Functional Requirements

1. **Privacy:** no video or landmarks are uploaded; messages travel peer-to-peer.
2. **Safety:** unsupported or uncertain gestures must not be sent automatically as words.
3. **Responsiveness:** recognition must not freeze the call interface.
4. **Accessibility:** keyboard access, visible focus, reduced-motion support and no information shown by colour alone.
5. **Reliability:** calls must reconnect after temporary network or server interruptions.
6. **Security:** hashed passwords, HTTP-only session cookies, rate limiting and security headers.

CHAPTER 5

SYSTEM REQUIREMENTS SPECIFICATION

5.1 SOFTWARE REQUIREMENTS

1. **Operating System:** Windows 10/11, macOS or Linux (development on Windows 11).
2. **Programming Languages:** TypeScript (frontend), Python 3.11 (backend and machine learning).
3. **Frontend:** Next.js 16, React 19, Tailwind CSS 4, shadcn/ui components.
4. **Backend:** FastAPI, SQLAlchemy 2 (async) with asyncpg, Alembic, Pydantic, PyJWT, argon2.
5. **Database:** PostgreSQL 17.
6. **Libraries and Packages:**
   - Machine learning: PyTorch, NumPy, scikit-learn, MLflow, ONNX.
   - In-browser inference: ONNX Runtime Web, MediaPipe Tasks Vision.
   - Testing: pytest, Node.js test runner, Playwright (headless Chrome).
7. **Browser:** Google Chrome or Microsoft Edge (WebRTC, WebAssembly, WebGL and Web Workers).
8. **Tools:** VS Code, Git, npm, pip, Claude Code.

5.2 HARDWARE REQUIREMENTS

9. **For users:** any laptop or phone able to run Chrome or Edge, with a webcam for signing and a microphone and speaker for speech.
10. **For development and training:** a laptop with 16 GB RAM; the project used an MSI laptop with an NVIDIA RTX 4060 (8 GB) GPU.
11. **Server:** a small machine for FastAPI and PostgreSQL; no GPU is required because recognition runs on users' devices.
12. **Network:** broadband or 4G; a TURN relay is recommended for calls across restrictive networks.

5.3 SOFTWARE DESCRIPTION

Next.js and React provide the user interface with the App Router, client components for the call and recognition, and a production build that serves static pages efficiently. **TypeScript** keeps the large client codebase type-safe, which matters because the browser re-implements the exact feature computation used in training.

FastAPI serves the REST API for accounts and meetings and a WebSocket endpoint for call signalling. SQLAlchemy with asyncpg accesses **PostgreSQL**, and Alembic manages schema migrations. Passwords are hashed with **argon2** (Biryukov et al., 2016) and sessions are signed JSON Web Tokens stored in HTTP-only cookies.

PyTorch (Paszke et al., 2019) is used to train the sign models, **MLflow** records every training run, and the trained model is exported to **ONNX**. In the browser, **ONNX Runtime Web** executes the model with WebAssembly and **MediaPipe Tasks Vision** runs the pose, hand and face landmark models on the GPU through WebGL.

5.4 DATASET DESCRIPTION

The INCLUDE dataset provides all training and evaluation data. Clips of the 39 supported words were downloaded together with three clips each of the 223 other INCLUDE words, which serve as real examples of unsupported signs. Landmarks were extracted with the same in-browser extractor that the application uses, by running the import page in headless Chrome, so that training and live features come from identical code. An automated audit found no corrupt videos, no duplicate files and no duplicate landmark sequences, and flagged one clip with too little hand tracking, which is excluded automatically.

Table 2 Dataset Summary

Subset	Train	Validation	Test	Total
Supported signs (39 words), official split	533	54	121	708
Supported signs, session-held-out split	476	122	110	708
Unknown signs (223 other words)	375	153	141	669
Unknown-sign words (word-disjoint)	125	51	47	223

Unknown-sign words are split by word, so the 47 words in the test set are never seen during training; the test therefore measures rejection of genuinely new signs. Because 57 of the 74 recording sessions in INCLUDE's official split appear in more than one subset, a stricter session-held-out split was also created to give a more honest estimate of accuracy.

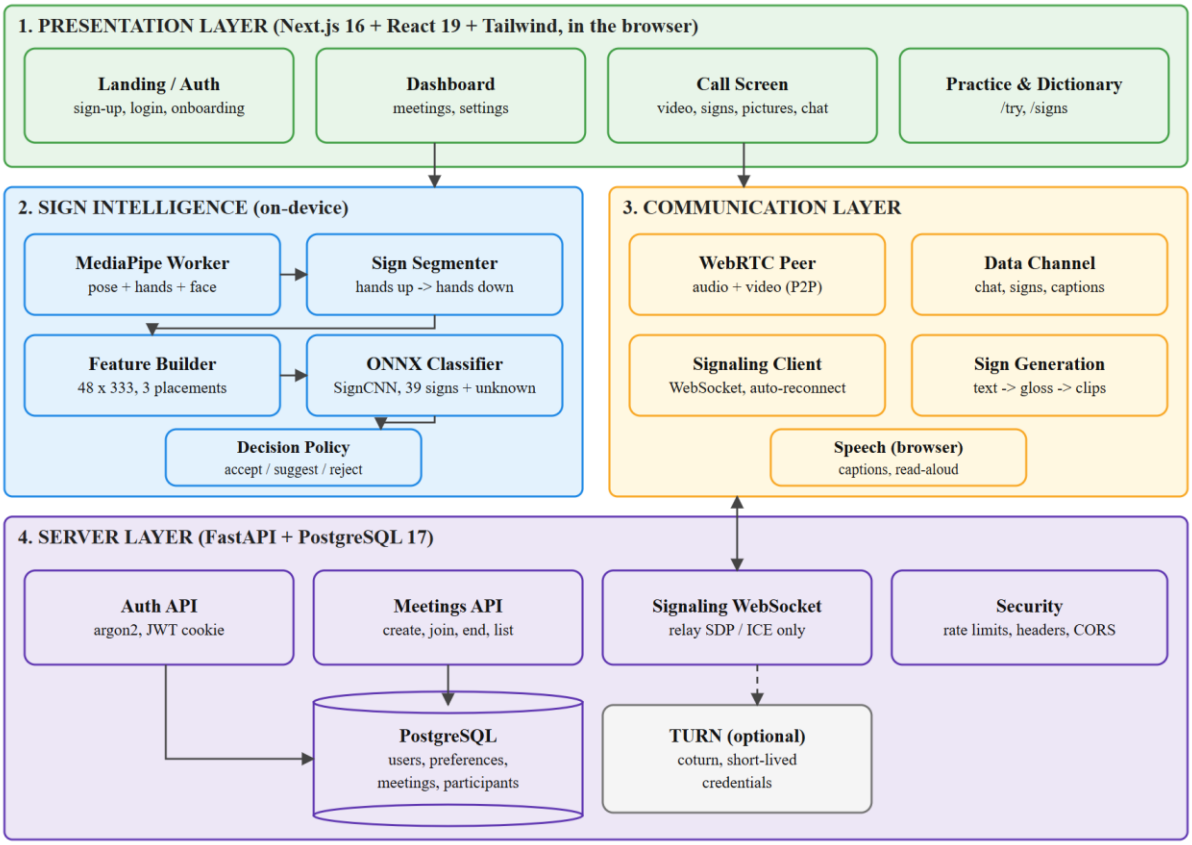
CHAPTER 6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

The SignSense architecture is organised into four layers that are kept deliberately independent. The **Presentation Layer** contains the web pages: the landing page, sign-up and onboarding, the dashboard, the call screen, the practice page and the sign dictionary. The **Sign Intelligence Layer** runs entirely in the browser: a Web Worker executes MediaPipe, a segmenter detects individual signs, a feature builder produces the model input, the ONNX classifier scores it and a decision policy chooses what to do with the result.

The **Communication Layer** manages the WebRTC peer connection for audio and video, the data channel for chat, captions and recognised signs, the signalling client with automatic reconnection, the text-to-sign generator and the browser's speech services. The **Server Layer** is a FastAPI application with authentication, meeting management, a signalling WebSocket that only relays connection set-up messages, and security controls, backed by PostgreSQL. An optional TURN server relays media when a direct connection is impossible. A failure in recognition never ends the call, because the call does not depend on it.

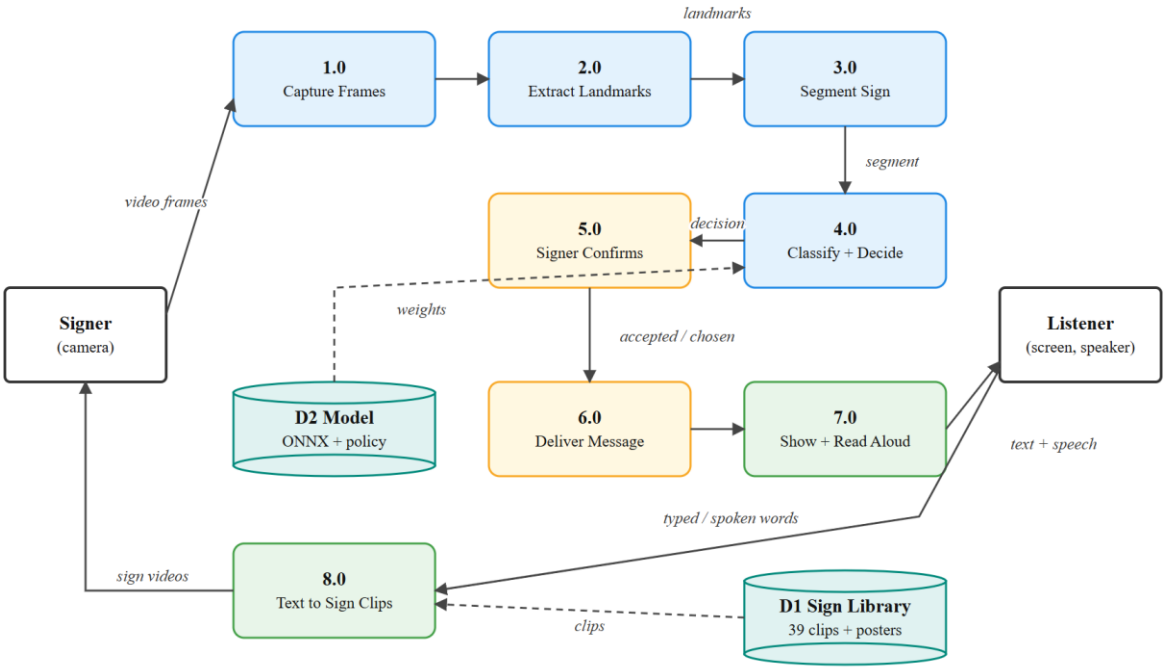


6.1 System Architecture

6.2 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) describes how information moves between the signer, the processes and the listener:

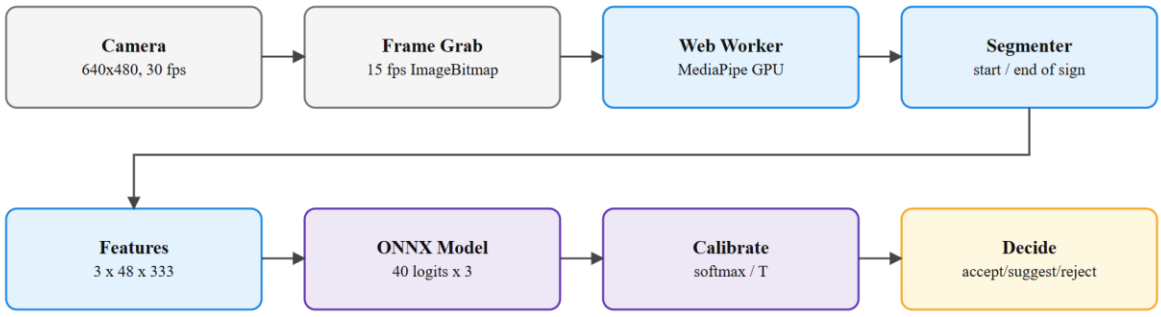
1. **Capture Frames:** the signer's camera stream, shared with the call, is sampled at 15 frames per second.
2. **Extract Landmarks:** the worker converts each frame into hand, body and face landmarks.
3. **Segment Sign:** frames between hands-up and hands-down are collected as one sign.
4. **Classify and Decide:** the model (data store D2) scores the sign and the policy decides accept, suggest or reject.
5. **Signer Confirms:** accepted signs are sent after a cancellable countdown; suggested signs need a tap.
6. **Deliver and Present:** the word is sent over the data channel, shown to the listener and read aloud.
7. **Text to Sign:** the listener's typed or spoken words are mapped to clips from the sign library (D1) for the signer.



6.2 Data Flow Diagram

6.3 LIVE RECOGNITION PIPELINE

Figure 6.3 shows the recognition pipeline in detail. The camera track that is already used for the call is attached to a hidden video element; a timer grabs a frame every 66 ms as an ImageBitmap and transfers it to the Web Worker. Only one frame is processed at a time: if the worker is still busy, the new frame is skipped rather than queued, so the pipeline never builds up a backlog. Features are resampled by timestamp, so skipped frames do not distort the speed of a sign.

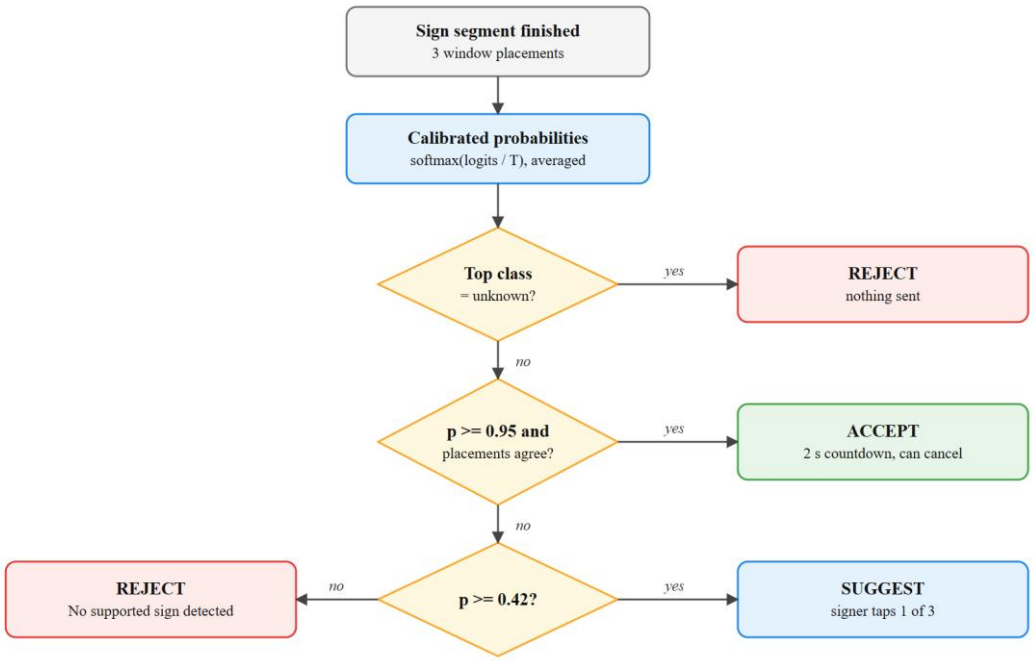


Everything above runs inside the signer's browser; only the final text is sent to the other person.

6.3 Live Recognition Pipeline

6.4 DECISION POLICY

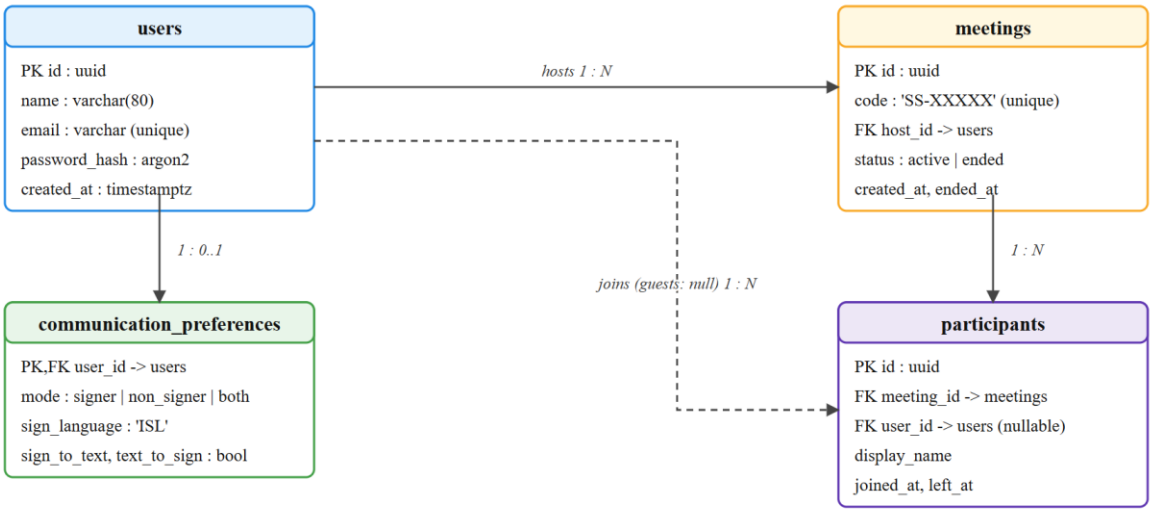
The decision policy turns the model's scores into one of three actions. Probabilities are first calibrated with a temperature fitted on validation data and averaged over three placements of the 48-frame window. If the most likely class is 'unknown', the sign is rejected. A sign is accepted for automatic sending only if its probability is at least 0.95 and all three window placements agree on it; because probabilities sum to one, this also guarantees a lead of at least 0.90 over the second-best class. Signs between 0.42 and that level are offered as a choice of three, and everything else is rejected. All thresholds were chosen on validation data, not on the test set.



6.4 Decision Policy

6.5 DATABASE DESIGN

The database stores only what the service needs: user accounts, communication preferences, meetings and participation records. No video, landmarks or message contents are stored. Deleting an account removes the user, their preferences and the meetings they hosted; in other people's meeting history the user's name is replaced with 'Deleted user'.



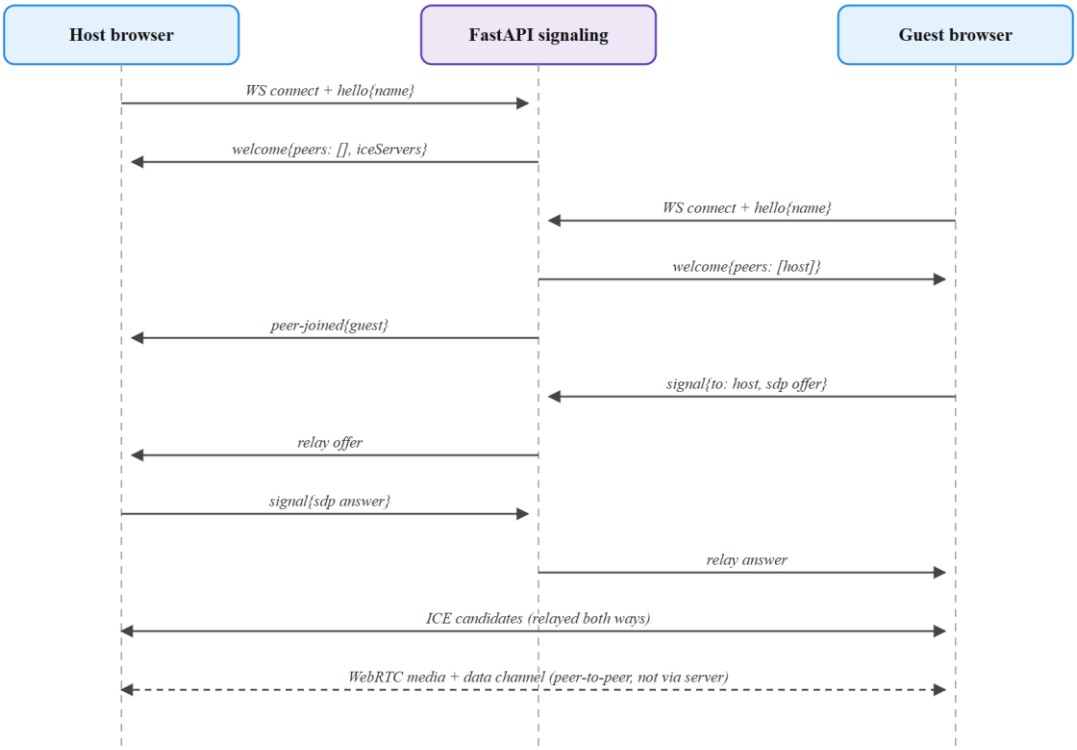
6.5 Entity Relationship Diagram

Table 3 Backend API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/signup, /api/auth/login	Create an account / sign in (rate-limited); sets the session cookie
POST	/api/auth/logout	Clear the session cookie
GET	/api/auth/me	Current user and preferences
PATCH / DELETE	/api/users/me	Rename the account / delete it (password required)
PUT	/api/users/me/preferences	Communication mode and ISL options
POST / GET	/api/meetings	Create a meeting / list the user's recent meetings
GET	/api/meetings/{code}	Look up a meeting by code (guests allowed, rate-limited)
POST	/api/meetings/{code}/end	Host ends the meeting for everyone
WS	/ws/meetings/{code}	Signalling: hello, signal (SDP / ICE relay), leave, ping

6.6 CALL SIGNALLING SEQUENCE

Figure 6.6 shows how two browsers establish a call. Each browser opens the signalling WebSocket and introduces itself with a hello message. The newcomer receives the list of people already present and sends them an SDP offer through the server; the answer and the ICE candidates are relayed in the same way. Once the connection is established, audio, video and the data channel flow directly between the browsers. If the signalling connection drops, the client reconnects with exponential backoff and re-offers, and the server frees seats whose sockets have been silent for 70 seconds.



6.6 Call Signalling Sequence

6.7 USER INTERFACE DESIGN

The interface follows a single design system: a light, warm background for the website and a dark workspace for the call, one deep-blue brand colour with a warm orange accent, Bricolage Grotesque headings with Geist body text, and large touch targets. The call screen places six controls at the bottom (microphone, camera, signs, pictures, chat and leave); chat and the picture board open as a side panel on wide screens and as a bottom sheet on phones. The recognition status and the confirmation card appear in their own strip below the video, so they never cover either person. Animations use only opacity and transforms and are switched off when the user's system requests reduced motion.

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 MODULES

- Authentication, Preferences and Meetings
- Video Calling and Signalling
- Landmark Extraction (Web Worker)
- Sign Segmentation and Feature Engineering
- Sign Classification Model and Training
- Calibration and Decision Policy
- Text-to-Sign Generation, Picture Board and Captions
- Automated ML Pipeline and Evaluation

7.2 MODULE DESCRIPTION

7.2.1 Authentication, Preferences and Meetings

Implemented in backend/app/api/auth.py and meetings.py. Users sign up with a name, email and password; the password is hashed with argon2, and a signed JWT is stored in an HTTP-only, SameSite=Lax cookie. Login attempts, sign-ups, meeting look-ups and signalling joins are rate-limited per client. Each user stores a communication mode (signer, non-signer or both) that decides which tools a call opens with. Meetings receive a short code such as SS-7F4K2 drawn from an alphabet without ambiguous characters; the code is accepted even when typed carelessly. Meetings with nobody connected expire after 24 hours, and the host can end a meeting for everyone. In production mode the server refuses to start with unsafe settings such as insecure cookies or a short secret.

7.2.2 Video Calling and Signalling

Implemented in frontend/hooks/use-call.ts and backend/app/api/signaling.py. The browser creates an RTCPeerConnection, adds the camera and microphone tracks with a 'maintain frame rate' preference (smooth motion matters more than resolution for signing) and opens a data channel named 'app'. The signalling server relays SDP offers, answers and ICE candidates between the two participants and never inspects media. The client reconnects automatically after a dropped WebSocket, restarts ICE after a failed connection, and the server issues short-lived TURN credentials (HMAC of an expiry time and the peer id) when a TURN server is configured.

7.2.3 Landmark Extraction

Implemented in frontend/lib/landmarks/landmark-worker.ts and extractor.ts. The worker runs three MediaPipe models on the GPU: pose-lite on the full frame, the hand model on a square crop around the upper body, and the face model on a crop around the head, which keeps small hands and faces detectable. Detected hands are assigned to the person's left and right by matching them to the pose wrists. Running MediaPipe in a worker removed a 6.75-second freeze that its first GPU frame used to cause on the call screen; the worker warms the models up on a blank frame while loading.

7.2.4 Sign Segmentation and Feature Engineering

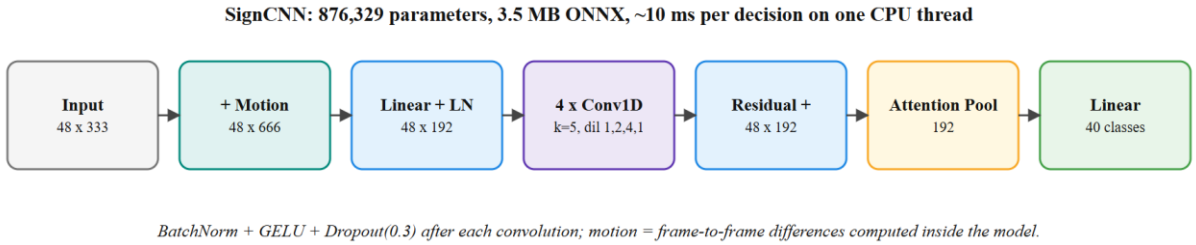
Implemented in frontend/lib/signs/segmenter.ts and features.ts, with an identical Python implementation in ml/signsense_ml. A sign starts when a hand rises above a rest line placed 1.3 shoulder-widths below the shoulders, and ends 450 ms after the hands drop. The segment keeps 300 ms before and 250 ms after the movement, must last at least 350 ms, and is cut at 4 s; after a cut, the hands must come down before another sign can start, so holding a pose does not repeat a word. Each frame becomes a 333-value feature vector expressed in a body frame:

$$x' = (x \cdot a - m) / s\,, \quad y' = (y - m_y) / s$$

where a is the camera aspect ratio, m is the midpoint of the shoulders and s is the shoulder width, so the features do not depend on where the person sits or how far they are from the camera. The vector contains 9 pose points, both hands in the body frame (2 × 21 points), both hands relative to their wrist and size (hand shape), 72 face points and three presence flags. The sequence is resampled to 15 frames per second and fitted into a 48-frame window (3.2 s). Python and browser features are checked to match within 10^{-6} by a parity test.

7.2.5 Sign Classification Model and Training

Implemented in ml/signsense_ml/model.py and train.py. SignCNN first appends frame-to-frame differences (motion) to the features, projects them to 192 dimensions, applies four dilated 1-D convolutions (kernel 5, dilations 1, 2, 4, 1) with batch normalisation, GELU and dropout, adds a residual connection, pools over time with learned attention weights and classifies into 40 classes (39 signs and 'unknown'). A bidirectional GRU baseline reached the same accuracy with 1,255,144 parameters, so the smaller and faster CNN was chosen.



7.1 SignCNN Model Architecture

Table 4 Training Configuration

Setting	Value
Optimizer	AdamW, learning rate 1e-3, weight decay 1e-2 (Loshchilov & Hutter, 2019)
Schedule	One-cycle learning rate (Smith & Topin, 2019), up to 120 epochs, early stopping after 40
Loss	Cross-entropy with label smoothing 0.1 and square-root inverse-frequency class weights
Batch size	32 sequences
Augmentation	Rotation, scaling, stretch and shift around the shoulders; landmark jitter; random window position
Model selection	Best validation macro-F1
Export	ONNX (3.5 MB), checked by Python-to-browser logit parity tests

7.2.6 Calibration and Decision Policy

Implemented in ml/signsense_ml/policy.py and frontend/lib/signs/decision.ts. The logits z of each window placement are converted into calibrated probabilities with temperature T, and the three placements are averaged:

$$p_k = (1/3) \Sigma_v \exp(z_{v,k} / T) / \Sigma_j \exp(z_{v,j} / T)$$

T = 0.89 was fitted on validation data by minimising the negative log-likelihood. The accept threshold (0.95) and suggest threshold (0.42) were then chosen on validation data so that at most 2% of unknown validation signs could be sent automatically and automatic sends were at least 97% precise. The browser implementation reproduces the Python decisions exactly, which is verified by running real test clips through the published model in ONNX Runtime Web.

7.2.7 Text-to-Sign Generation, Picture Board and Captions

Implemented in frontend/lib/signs/gloss.ts, components/signs/sign-player.tsx, components/saathi/*, ml/blender/saathi3d.py and components/call/phrase-board.tsx. Incoming text is normalised, multi-word signs such as 'how are you' and 'good morning' are matched first, synonyms are mapped to dictionary entries, and the resulting signs are performed one after

another by Saathi, an animated signer shown in 3D (rendered in Blender on the GPU) or as a 2D drawing. Her motion is computed from the body, hand and face landmarks of one INCLUDE recording per sign, so no real signer's video or photo appears on the website. Run through MediaPipe and the published model, the 3D Saathi was read as the intended sign first for 35 of the 39 signs and the 2D drawing for 33 (none accepted wrongly), against 39 of 39 for the source landmarks. Words without a sign are shown as large text and are never fingerspelled or guessed. The picture board offers eighteen one-tap messages with icons. Live captions use the browser's speech recognition and are off by default, because Chrome sends the audio to Google; read-aloud uses on-device speech synthesis.

7.2.8 Automated ML Pipeline and Evaluation

Implemented in ml/signsense_ml/pipeline.py. One resumable command checks the environment, optionally downloads INCLUDE clips, extracts landmarks in headless Chrome, audits the data, trains a candidate model, evaluates the candidate and the published model on the same test clips, and publishes the candidate only if it sends fewer wrong words and loses at most two points of accuracy. Progress is saved after every step, so an interrupted run continues where it stopped.

7.2.9 Algorithm

1. Acquire the camera stream once and share it between the call and recognition.
2. Every 66 ms, grab a frame and send it to the Web Worker unless the previous frame is still being processed.
3. Extract pose, hand and face landmarks in the worker and return them to the page.
4. Feed each landmark frame to the segmenter; when the hands drop, emit the frames of one sign.
5. Resample the sign to 15 fps and build the 48 × 333 features for three window placements.
6. Run the ONNX model on the three placements and calibrate the logits with temperature T.
7. Average the probabilities and apply the decision policy (reject, accept or suggest).
8. For accept, start a two-second cancellable countdown; for suggest, wait for the signer's choice.
9. Send the chosen word once over the data channel and confirm 'Sent' or 'Not sent' to the signer.
10. On the receiving side, validate the message, show it as a caption, add it to the transcript and read it aloud.

CHAPTER 8

SYSTEM TESTING AND MAINTENANCE

8.1 UNIT TESTING

Unit tests check individual functions in isolation. The backend has 66 pytest tests running against a separate test database, the machine-learning package has 11 pytest tests, and the browser logic has 29 tests run with the Node.js test runner. All of them pass. Table 5 lists representative cases.

Table 5 Representative Unit Test Cases and Outcomes

No.	Test Case	Purpose	Result
1	test_signup_sets_session_and_me_works	Sign-up stores a hashed password and sets an HTTP-only cookie	Pass
2	test_login_is_rate_limited	Repeated wrong passwords are blocked	Pass
3	test_two_peers_meet_and_relay_signals	Signalling relays offers between two peers	Pass
4	test_third_person_is_refused	A 1:1 meeting rejects a third participant	Pass
5	test_host_ending_meeting_disconnects_everyone	Ending a meeting closes every connection	Pass
6	test_turn_credentials_are_per_peer_and_expire	TURN credentials are per peer and never expose the secret	Pass
7	test_delete_account_needs_password_and_removes_everything	Account deletion requires the password and cascades	Pass
8	test_settings_refuse_weak_secret_and_unsafe_production	Unsafe production settings stop the server	Pass
9	segmenter: hands held up give one segment	No repeated words while hands stay raised	Pass
10	published model reproduces Python logits and decisions	Browser inference matches training exactly	Pass
11	published policy: every auto-send has a large lead	Accepted signs always agree and lead by >= 0.9	Pass
12	malformed or hostile sign messages are rejected	Received messages are validated	Pass

8.2 INTEGRATION TESTING

Integration testing checks that modules work together. A feature-parity test runs nine recorded clips through both the Python and TypeScript feature code and confirms identical output. An inference test runs twelve real test clips, including four unsupported signs, through the published ONNX model in ONNX Runtime Web and checks that every logit is within 2×10^{-3} of Python and every decision matches. A migration test creates a fresh database, applies all

Alembic migrations, compares the result with the SQLAlchemy models (0 differences) and rolls them back.

8.3 FUNCTIONAL TESTING

Functional testing verifies complete user journeys in a real browser against the real backend and database. Eleven automated journeys covered the landing page, sign-up, onboarding, the dashboard, meetings, settings, account deletion, the dictionary and links; all eleven passed.

Table 6 Functional Test Journeys

No.	Journey	Checks	Result
1	Landing page navigation	Smooth anchor scrolling below the sticky header, scroll reveal, call-to-action	Pass
2	Reduced motion	No hidden content and no smooth scrolling when motion is reduced	Pass
3	Mobile menu	Opens, closes with Escape, returns focus, navigates	Pass
4	Sign-up validation	Field errors, focus on first error, show password, account created	Pass
5	Onboarding	Preferences saved; dashboard guide reflects real state	Pass
6	Start a meeting	New meeting opens the pre-join screen with an invite link	Pass
7	Settings	Rename with confirmation	Pass
8	Delete account	Wrong password refused; correct password deletes the account	Pass
9	Login error	Wrong password message; input kept	Pass
10	Dictionary	Deep link filter, clip playback, words without a sign reported	Pass
11	Links	52 internal links checked; none broken	Pass

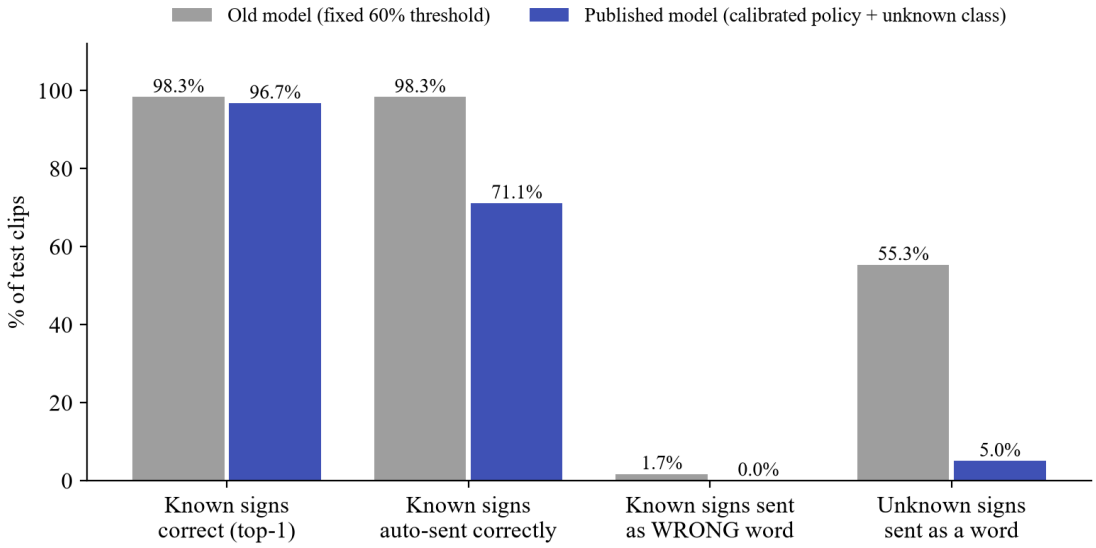
8.4 SYSTEM TESTING

8.4.1 Model Evaluation

The published model was compared with the earlier model on the same official INCLUDE test clips: 121 clips of supported signs and 141 clips of 47 unsupported words that were never seen in training. Table 7 shows that the new model sends no wrong word for supported signs and reduces unknown signs sent as words from 78 of 141 to 7 of 141, at the cost of two clips of top-1 accuracy and more confirmations by the signer.

Table 7 Comparison of the Earlier and Published Models (Official Test Split)

Measure	Earlier model	Published model
Supported signs: top-1 accuracy	98.3% (119/121)	96.7% (117/121)
Supported signs: top-3 accuracy	100%	99.2%
Supported signs sent automatically and correct	119	86
Supported signs sent automatically as the WRONG word	2	0
Supported signs offered as a choice / rejected	0 / 0	31 / 4
Unknown signs sent automatically as a word	78 / 141 (55.3%)	7 / 141 (5.0%)
Unknown signs offered as a choice / rejected	-	21 / 113
Decision rule	fixed 60% threshold	T = 0.89, accept >= 0.95, suggest >= 0.42
Latency per decision (1 CPU thread)	about 10 ms	about 10 ms

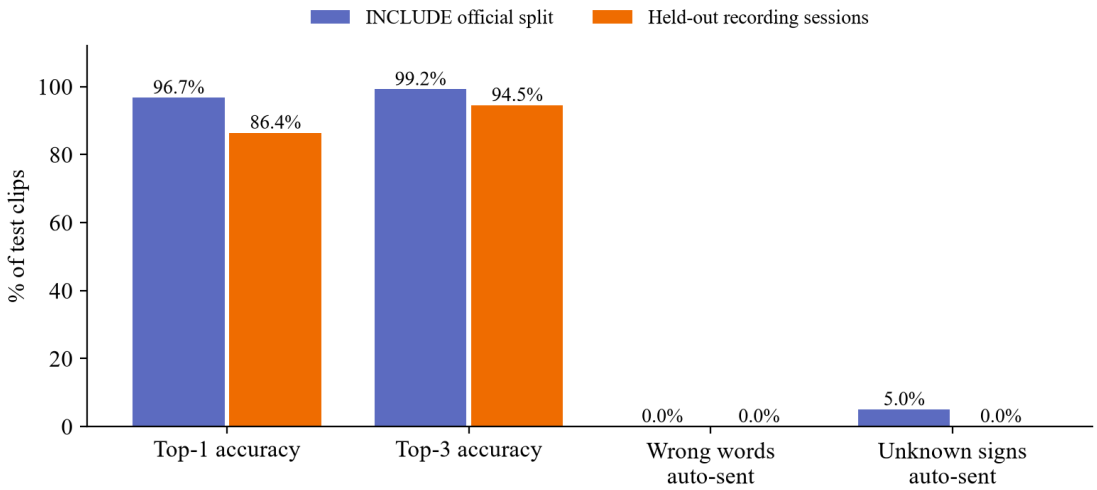


8.1 Earlier versus Published Model

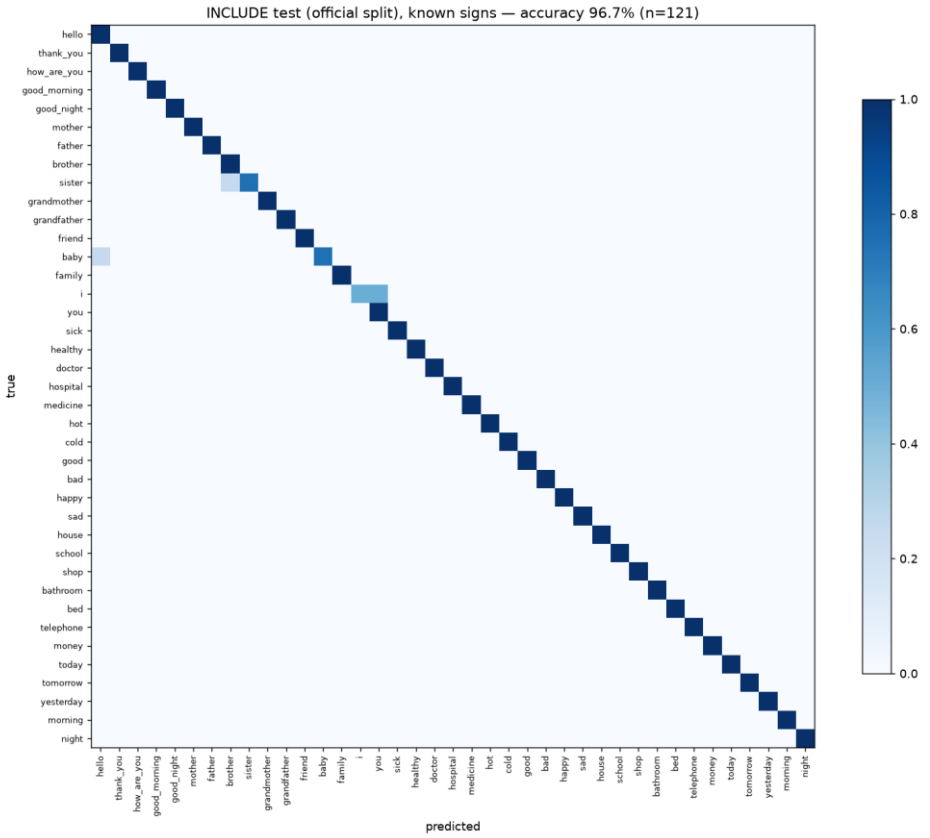
INCLUDE's official split places clips from the same recording sessions in both training and test data, so its results are optimistic. A model trained with the same recipe on a session-held-out split reached 86.4% top-1 and 94.5% top-3 accuracy; it sent 16 of 110 signs automatically, all correctly, offered the right sign among three choices for 80 more, and sent no unknown sign as a word. In other words, on new recordings the system rarely says something wrong but asks the signer to confirm more often.

Table 8 Official versus Session-Held-Out Evaluation

Measure	Official split	Held-out sessions
Supported-sign test clips	121	110
Top-1 / top-3 accuracy	96.7% / 99.2%	86.4% / 94.5%
Macro-F1	97.3%	81.0%
Sent automatically: correct / wrong	86 / 0	16 / 0
Offered as a choice (right sign included)	31	80
Rejected	4	13
Unknown signs sent automatically	7 / 141	0 / 141



8.2 Official versus Session-Held-Out Results



8.3 Confusion Matrix of the Published Model

The confusion matrix shows that almost all errors involve signs that point at the body. The most common confusion is between 'I' and 'you', and an unseen sign for 'alive' was accepted as 'I'; both are pointing gestures whose difference is mainly depth, which two-dimensional landmarks capture poorly.

8.4.2 Ablation Study

To understand which inputs matter, models were retrained with parts of the features removed (three seeds each, official split). The results show that hand-shape features help most, while the official test set is too easy to separate the other variants; one test clip equals 0.83 percentage points.

Table 9 Ablation Results (Mean of Three Seeds)

Variant	Accuracy %	Macro-F1 %	Top-3 %
Full model (CNN)	98.3	98.6	100.0
GRU instead of CNN	98.3	98.6	100.0
Without augmentation	98.3	98.6	100.0
Without face	98.3	98.6	100.0
Without body pose	97.8	98.2	100.0
Without hand-shape features	97.0	97.4	100.0
Hands only	98.1	98.4	100.0

8.4.3 Rejection Study

Four additional rejection rules were evaluated to see whether the 7 false acceptances could be reduced without lowering any safeguard: requiring every window placement to exceed a threshold, limiting the probability of 'unknown', an energy score and per-class thresholds. Thresholds were chosen on validation data and scored once on the same test set. None reduced the test false acceptances, and the per-class rule cost four correct automatic sends, so the published policy was kept unchanged. Three of the seven false acceptances were predicted as 'yesterday'.

Table 10 Rejection Rules Evaluated (Official Test Split)

Rule	Unknown sent (of 141)	Supported sent correctly (of 121)
Published policy	7	86
Every placement above threshold	7	86
Probability of unknown below threshold	7	86
Energy score above threshold	7	86
Per-class thresholds	7	82

8.4.4 End-to-End Call Testing

The complete workflow was tested with two headless Chrome browsers in a real call. Chrome's fake camera device played recorded INCLUDE test clips through the application's real camera code, so landmark extraction, segmentation, the model, the policy, the data channel and the receiving interface all ran for real. These tests use recordings of INCLUDE signers; they are not a substitute for testing with people at physical webcams.

Table 11 End-to-End Call Tests

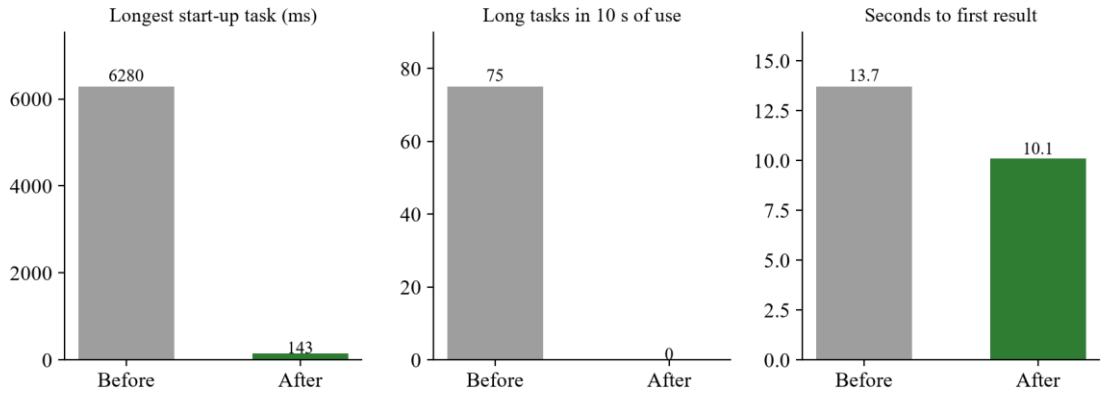
No.	Test	Observed Behaviour	Result
1	Camera start and errors	Live track; blocked or missing camera explained, joining still possible	Pass
2	Supported sign 'Hello'	Accepted three times; 2 sent = 2 received, unique ids	Pass
3	Unknown sign 'alive'	Rejected every time; nothing delivered	Pass
4	Unknown sign 'curved'	Rejected or offered as a choice; nothing delivered	Pass
5	Uncertain result	Not sent without a tap; chosen sign marked 'chosen by the signer'	Pass
6	Signs off and on	Off: no processing; on: exactly one loop at 15 frames/s	Pass
7	Model fails to load	Clear error; call and chat continue; Retry recovers	Pass
8	Corrupt model file	Plain error message with Retry	Pass
9	Leaving the call	All camera and microphone tracks stopped; no processing	Pass
10	Connection interruptions	Signalling restart and listener drop-out recovered; no duplicates	Pass
11	Phone layout (375 x 812)	Recognition card below the video; controls visible	Pass
12	Desktop layout (1440 x 900)	Recognition card below the video; controls visible	Pass

8.4.5 Performance

Performance was measured in headless Chrome on the development laptop. Moving MediaPipe into a Web Worker removed long main-thread tasks during recognition entirely; on the practice page the longest start-up task fell from 6,280 ms to 143 ms and the first result appeared after 10.1 s instead of 13.7 s.

Table 12 Performance Measurements

Measure	Value
Model (ONNX) loading	about 1.2-1.5 s
MediaPipe ready, including GPU warm-up in the worker	about 7.7 s (production build)
MediaPipe per frame (pose / hands / face)	about 35 ms (12 / 14 / 9 ms)
Frames processed	about 14.8 per second (target 15)
Decision for one sign (3 placements)	about 10 ms
Main-thread long tasks during 10 s of recognition	0 (previously 11)
JavaScript heap over 10 s	15 MB to 16 MB



8.4 Practice Page Before and After the Web Worker

8.5 WHITE BOX TESTING

White box testing examined internal code paths: the segmenter's start, end, maximum-length and rest rules; the decision policy's unknown, accept, suggest and reject branches, including unstable placements; the message validator's handling of missing, oversized and hostile fields; the recognition loop's error counter, skip-when-busy logic and cleanup; and the backend's rate limiters, idle timeout and meeting expiry. Each branch is covered by at least one automated test.

8.6 BLACK BOX TESTING

Black box testing evaluated behaviour only through inputs and outputs. Supported, unsupported and uncertain clips produced the expected cards and deliveries; invalid email addresses, short passwords and wrong passwords produced clear messages; a blocked camera produced instructions to fix it; and every page was rendered at 320, 375, 768 and 1440 pixels wide without horizontal scrolling. A malformed model file, a missing model and a server restart during a call were handled without ending the call.

8.7 SECURITY TESTING

Security tests confirmed that passwords are never returned, sessions use HTTP-only cookies, login, sign-up, look-ups and signalling joins are rate-limited, other websites cannot open the signalling WebSocket, and production mode refuses insecure settings without printing secrets.

MAINTENANCE

The three systems (communication, sign intelligence and sign generation) are kept in separate modules, so each can be changed independently. A new model is published only if it beats the current one on the same test data, and contract tests then confirm that features, logits and decisions still match. The automated browser checks can be re-run after every change.

CHAPTER 9

FUTURE ENHANCEMENT AND CONCLUSION

FUTURE ENHANCEMENTS

- **Real-User Evaluation:** Test with Deaf ISL users and their families at real webcams, and measure accuracy on signers who were not in the training data.
- **Larger Vocabulary:** Add more everyday and medical signs, prioritised by what families actually need.
- **Depth-Aware Features:** Use landmark depth and pointing direction to separate signs such as 'I' and 'you'.
- **More Unknown-Sign Data:** Train with more varied unsupported signs and idle movements to reduce false acceptances further.
- **Continuous Signing:** Move from isolated words towards short phrases with sequence models.
- **Deployment:** Run the Docker configuration on a server with HTTPS and a TURN relay for calls across networks.

CONCLUSION

SignSense 2.0 demonstrates that a privacy-preserving, two-way ISL communication tool can be built from open-source components and run in an ordinary web browser. Supported signs are recognised on the signer's own device and delivered as text and speech; spoken and typed words are returned as ISL clips and pictures. The project's main contribution is not only accuracy but safe behaviour: by calibrating confidence, training an explicit unknown class from real unsupported signs and letting the signer confirm uncertain results, the published model sends no wrong word for supported test signs and reduces unknown signs sent as words from 55.3% to 5.0%. The full workflow is covered by automated tests, from unit tests to two-browser calls.

The results also show clear limits. The vocabulary is small, recognition is word by word, accuracy on new recording sessions is lower than on the official split, and the system has not yet been evaluated with real users at physical webcams. These limits define the next steps: real-user testing with the Deaf community, a larger vocabulary and depth-aware features.

APPENDIX

A.1 SOURCE CODE

decision.ts

```
// Decision policy — TypeScript twin of ml/signsense_ml/policy.py (fixtures: npm run test:signs).
// Turns the model's logits for the three window variants into one of:
//   accept  -> confident: auto-send after a short, cancellable countdown
//   suggest -> unsure: show the top choices, the signer must pick (never sent automatically)
//   reject  -> "No supported sign detected" (unknown sign, or too uncertain): nothing is sent
// Self-contained (no runtime imports) so Node can run the tests directly.

export type Decision = "accept" | "suggest" | "reject";

export interface DecisionPolicy {
  temperature: number;
  acceptThreshold: number;
  minMargin: number;
  suggestThreshold: number;
  unknownLabel: string | null;
  windowVariants: number;
}

/** Models published before calibration existed behaved like this (auto-send at 0.6, never reject).
*/
export const LEGACY_POLICY: DecisionPolicy = {
  temperature: 1,
  acceptThreshold: 0.6,
  minMargin: 0,
  suggestThreshold: 0,
  unknownLabel: null,
  windowVariants: 1,
};

export function softmax(logits: ArrayLike<number>, temperature = 1): number[] {
  const z = Array.from(logits, (v) => v / temperature);
  const max = Math.max(...z);
  const e = z.map((v) => Math.exp(v - max));
  const sum = e.reduce((a, b) => a + b, 0);
  return e.map((v) => v / sum);
}

const argmax = (xs: number[]) => xs.reduce((best, v, i) => (v > xs[best] ? i : best), 0);

/** Mean calibrated probabilities over the variants, and whether all variants agree on the top
class. */
export function combine(variantLogits: number[][], policy: DecisionPolicy): { probs: number[];
stable: boolean } {
  const used = variantLogits.slice(0, Math.max(1, policy.windowVariants));
  const probs = used.map((l) => softmax(l, policy.temperature));
  const mean = probs[0].map((_, c) => probs.reduce((s, p) => s + p[c], 0) / probs.length);
  const tops = new Set(probs.map(argmax));
  return { probs: mean, stable: tops.size === 1 };
}

export function decide(probs: number[], stable: boolean, classes: string[], policy: DecisionPolicy):
Decision {
  const order = probs.map((p, i) => [p, i] as const).sort((a, b) => b[0] - a[0]);
  const [p1, top] = order[0];
  const p2 = order.length > 1 ? order[1][0] : 0;
  if (policy.unknownLabel !== null && classes[top] === policy.unknownLabel) return "reject";
  if (p1 >= policy.acceptThreshold && p1 - p2 >= policy.minMargin && stable) return "accept";
  if (p1 >= policy.suggestThreshold) return "suggest";
  return "reject";
}

export interface Ranked {
  label: string;
  prob: number;
}

/** Supported signs ranked by probability ("unknown" removed — it is never offered as a choice). */
export function rankKnown(probs: number[], classes: string[], policy: DecisionPolicy): Ranked[] {
  return probs
    .map((prob, i) => ({ label: classes[i], prob }))
    .filter((r) => r.label !== policy.unknownLabel)
    .sort((a, b) => b.prob - a.prob);
}
```

segmenter.ts

```
import type { LandmarkFrame } from "../landmarks/schema";

// Explicit sign segmentation (decisions.md #9): a sign starts when a hand rises into the signing
// zone and ends when the hands drop back to rest. Each finished segment is classified once, which
// matches the training data (one whole sign per clip) and keeps resting/idle hands from producing
words.

const L_SHOULDER = 11;
const R_SHOULDER = 12;

export interface SegmenterOptions {
  /** How long hands must be down before the sign counts as finished. */
  endAfterMs: number;
  /** Frames kept before the start / after the end (the signer at rest, like the training clips). */
  prerollMs: number;
  postrollMs: number;
  minSignMs: number;
  maxSignMs: number;
}

export const DEFAULT_SEGMENTER: SegmenterOptions = {
  endAfterMs: 450,
  prerollMs: 300,
  postrollMs: 250,
  minSignMs: 350,
  maxSignMs: 4000,
};

/** A hand is "signing" when it's visible and its wrist is above the rest line below the shoulders.
*/
export function isSigning(f: LandmarkFrame, aspect: number): boolean {
  if (!f.leftHand && !f.rightHand) return false;
  const p = f.pose;
  if (!p) return true; // no body seen: fall back to "hands visible"
  const ly = p[L_SHOULDER * 4 + 1];
  const ry = p[R_SHOULDER * 4 + 1];
  const shoulderWidth = Math.hypot((p[L_SHOULDER * 4] - p[R_SHOULDER * 4]) * aspect, ly - ry);
  const restLine = (ly + ry) / 2 + 1.3 * shoulderWidth;
  const raised = (hand: number[] | null) => !!hand && hand[1] < restLine;
  return raised(f.leftHand) || raised(f.rightHand);
}

export type SegmenterState = "idle" | "signing";

export class SignSegmenter {
  private buffer: LandmarkFrame[] = [];
  private start = -1; // t of first signing frame
  private lastActive = -1;
  /** After a too-long cut, hands must come down before another sign can start (no repeats while
held up). */
  private needRest = false;
  state: SegmenterState = "idle";
  readonly aspect: number;
  private opts: SegmenterOptions;

  // Plain fields (no parameter properties) so Node's type stripping can run this file in tests.
  constructor(aspect: number, opts: SegmenterOptions = DEFAULT_SEGMENTER) {
    this.aspect = aspect;
    this.opts = opts;
  }

  /** Feed one frame; returns the frames of a finished sign segment, or null. */
  push(f: LandmarkFrame): LandmarkFrame[] | null {
    this.buffer.push(f);
    const active = isSigning(f, this.aspect);

    if (this.state === "idle") {
      // Keep only the pre-roll while waiting.
      while (this.buffer.length > 1 && f.t - this.buffer[0].t > this.opts.prerollMs)
this.buffer.shift();
      if (this.needRest) {
        if (!active) this.needRest = false;
        return null;
      }
      if (active) {
        this.state = "signing";
        this.start = f.t;
        this.lastActive = f.t;
      }
      return null;
    }
```

```
    if (active) this.lastActive = f.t;
    const ended = f.t - this.lastActive >= this.opts.endAfterMs;
    const tooLong = f.t - this.start >= this.opts.maxSignMs;
    if (!ended && !tooLong) return null;

    const duration = this.lastActive - this.start;
    const cutoff = this.lastActive + this.opts.postrollMs;
    const segment = this.buffer.filter((x) => x.t <= cutoff);
    this.reset(f);
    this.needRest = tooLong && !ended;
    return duration >= this.opts.minSignMs ? segment : null;
  }

  /** Forget any partial sign (recognition paused/stopped, or the call changed). */
  clear() {
    this.state = "idle";
    this.start = -1;
    this.lastActive = -1;
    this.needRest = false;
    this.buffer = [];
  }

  private reset(last: LandmarkFrame) {
    this.state = "idle";
    this.start = -1;
    this.lastActive = -1;
    this.buffer = [last];
  }
}
```

model.py (SignCNN)

```
class AttentionPool(nn.Module):
    def __init__(self, dim: int):
        super().__init__()
        self.score = nn.Linear(dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:  # [B, T, D] -> [B, D]
        w = torch.softmax(self.score(x), dim=1)
        return (w * x).sum(dim=1)


def with_motion(x: torch.Tensor) -> torch.Tensor:
    """Append frame-to-frame differences (velocity); computed inside the model so ONNX includes
it."""
    d = x[:, 1:] - x[:, :-1]
    d = torch.cat([torch.zeros_like(x[:, :1]), d], dim=1)
    return torch.cat([x, d], dim=-1)


class SignCNN(nn.Module):
    """Alternative: temporal 1D convolutions (fast in the browser) -> attention pooling."""

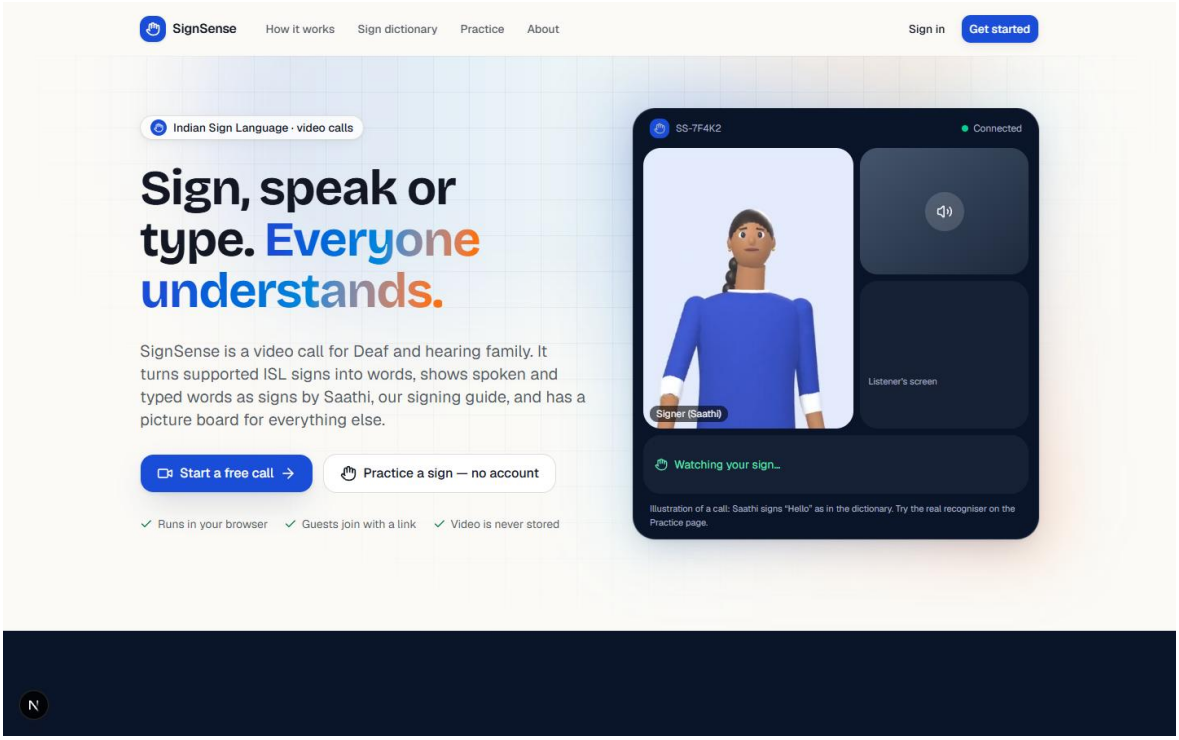
    def __init__(self, feature_dim: int, num_classes: int, hidden: int = 192, dropout: float = 0.3):
        super().__init__()
        self.proj = nn.Sequential(nn.Linear(feature_dim * 2, hidden), nn.LayerNorm(hidden),
nn.GELU())
        layers = []
        for dilation in (1, 2, 4, 1):
            layers += [
                nn.Conv1d(hidden, hidden, kernel_size=5, padding=2 * dilation, dilation=dilation),
                nn.BatchNorm1d(hidden),
                nn.GELU(),
                nn.Dropout(dropout),
            ]
        self.conv = nn.Sequential(*layers)
        self.pool = AttentionPool(hidden)
        self.head = nn.Sequential(nn.LayerNorm(hidden), nn.Dropout(dropout), nn.Linear(hidden,
num_classes))

    def features(self, x: torch.Tensor) -> torch.Tensor:
        """Pooled sequence representation (the classifier's input); used as a sign 'fingerprint'."""
        h = self.proj(with_motion(x))
        h = self.conv(h.transpose(1, 2)).transpose(1, 2) + h
        return self.pool(h)

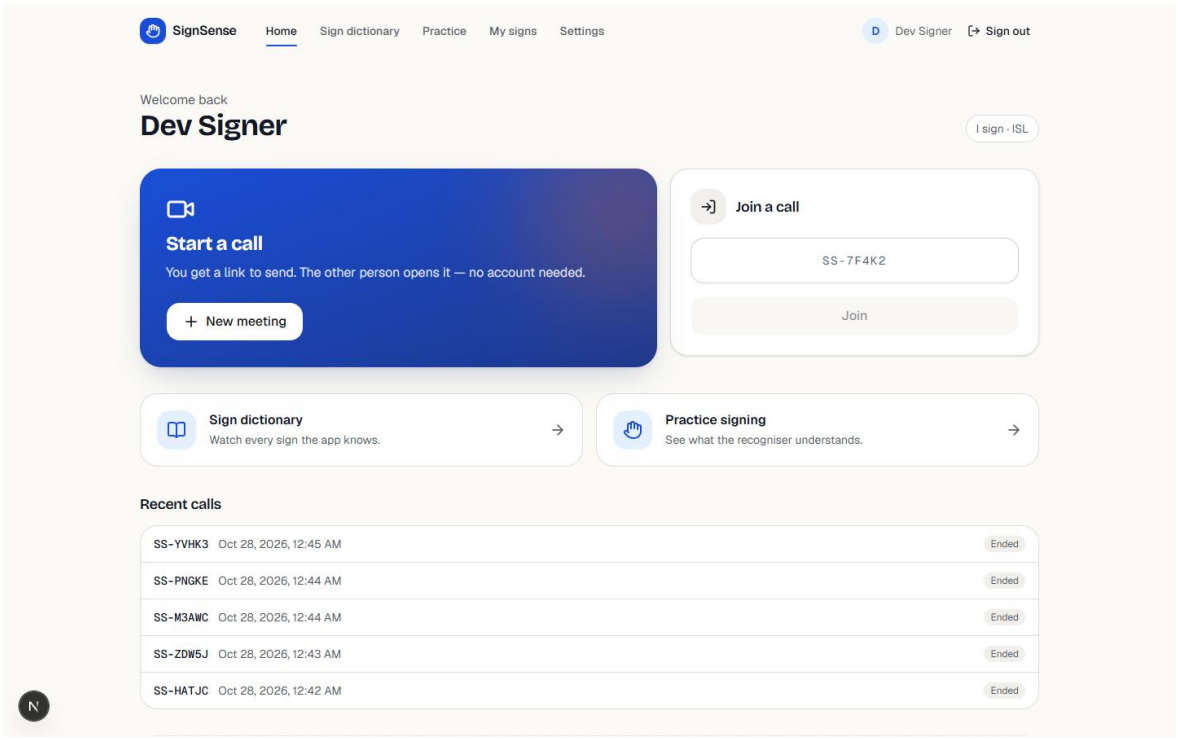
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))
```

A.2 SCREENSHOTS

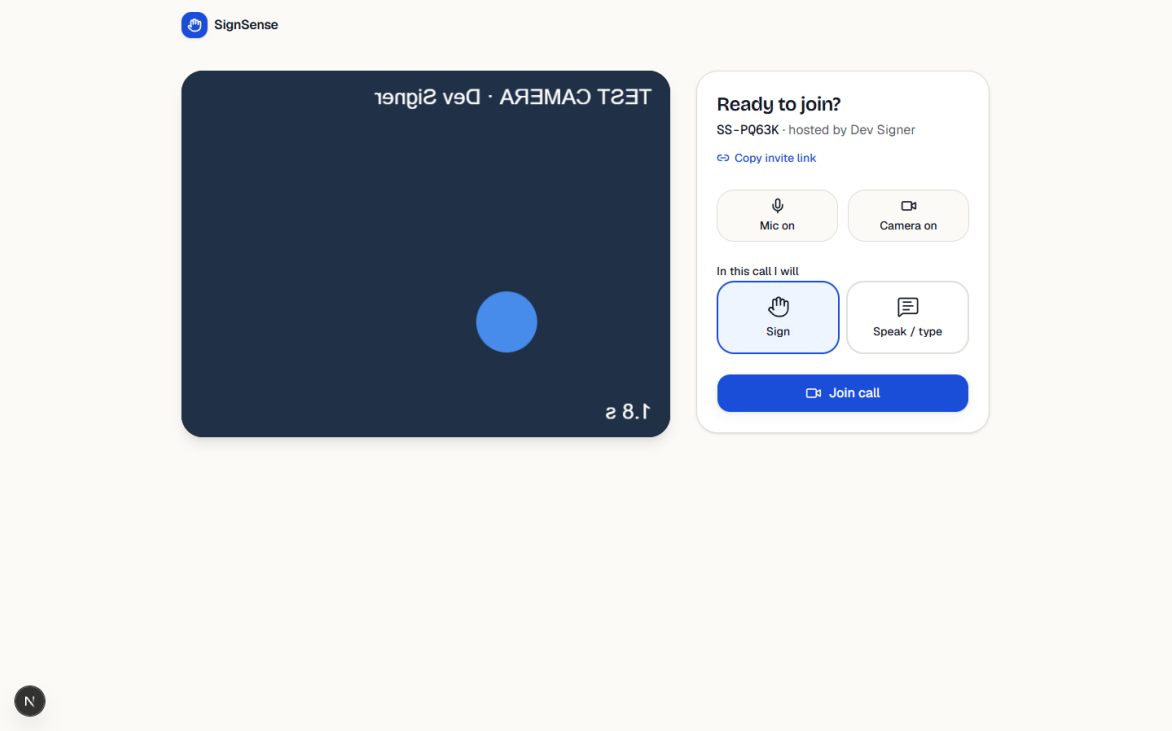
The following screenshots were captured automatically from the running application in headless Chrome (desktop 1440 x 900 and phone 375 x 812).



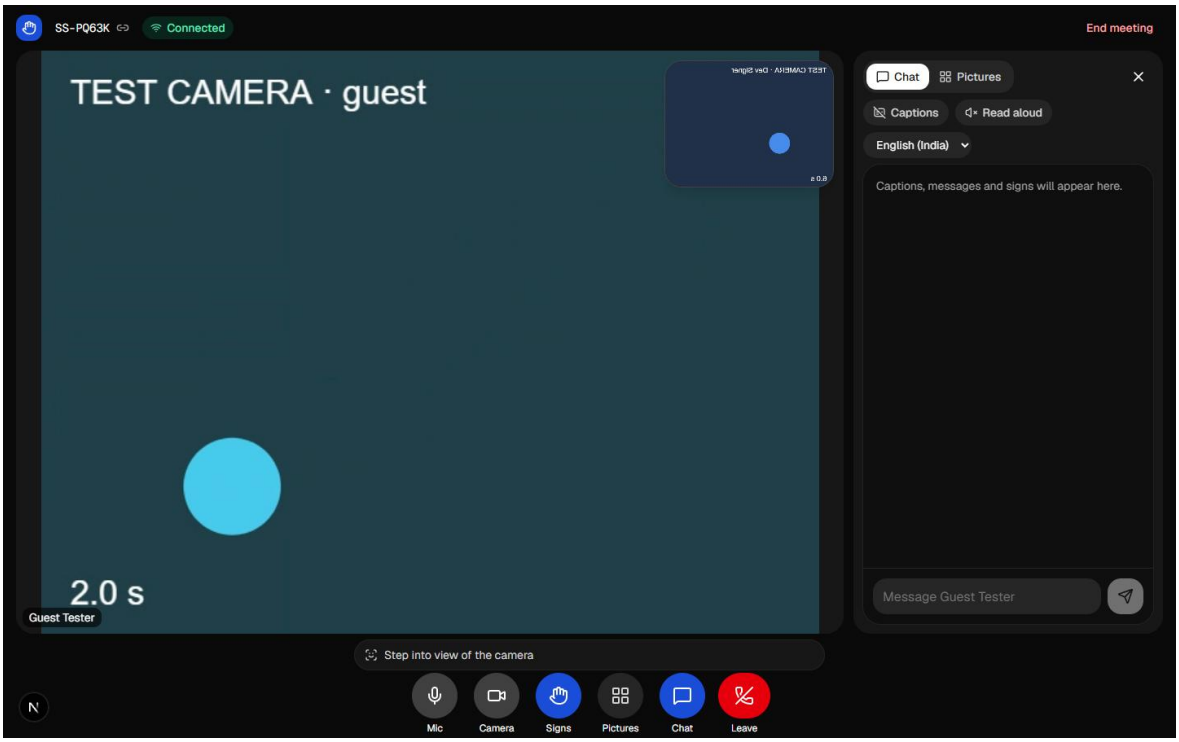
A.2.1 Landing Page



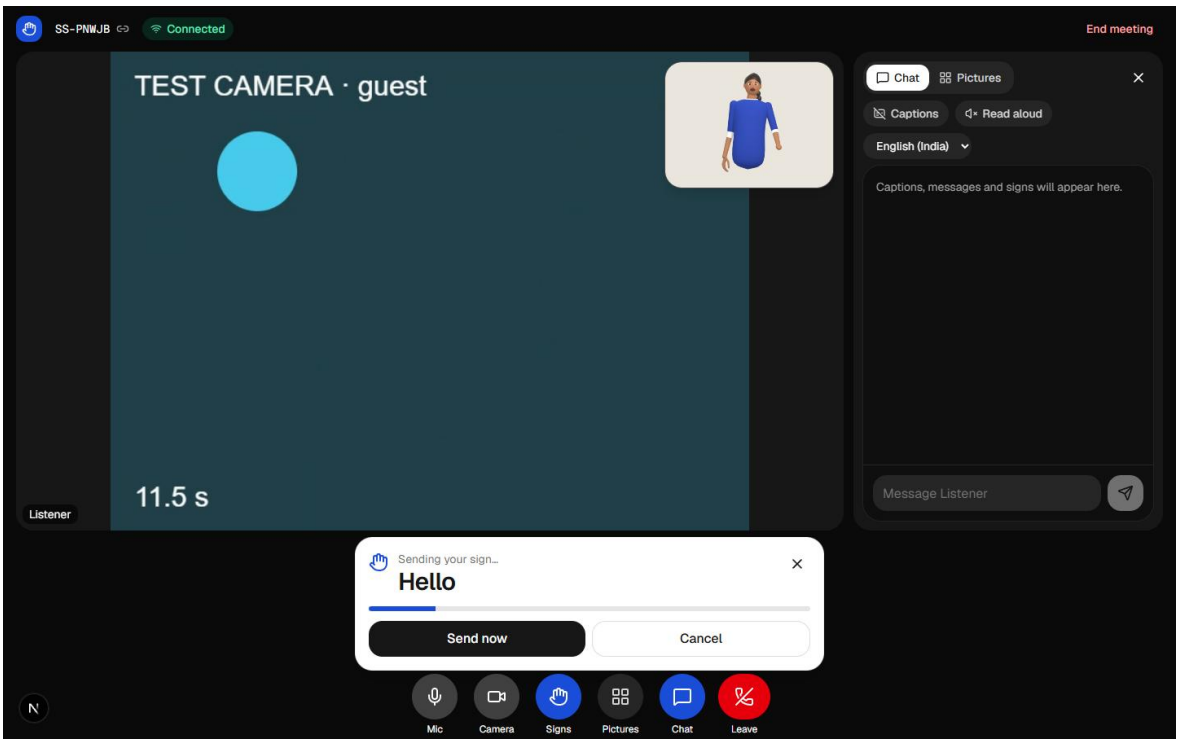
A.2.2 Dashboard



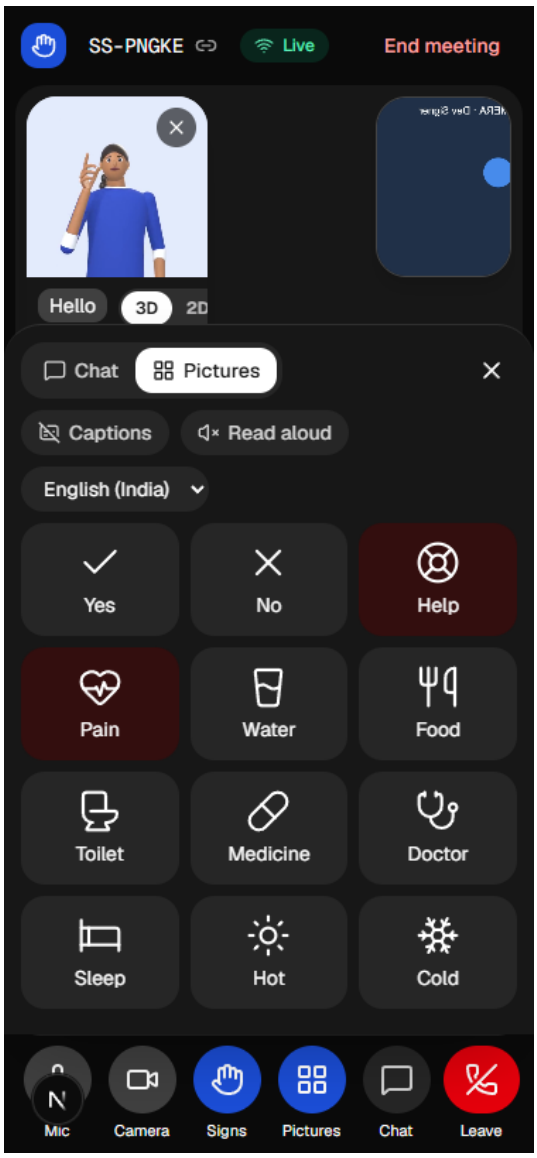
A.2.3 Pre-Join Screen



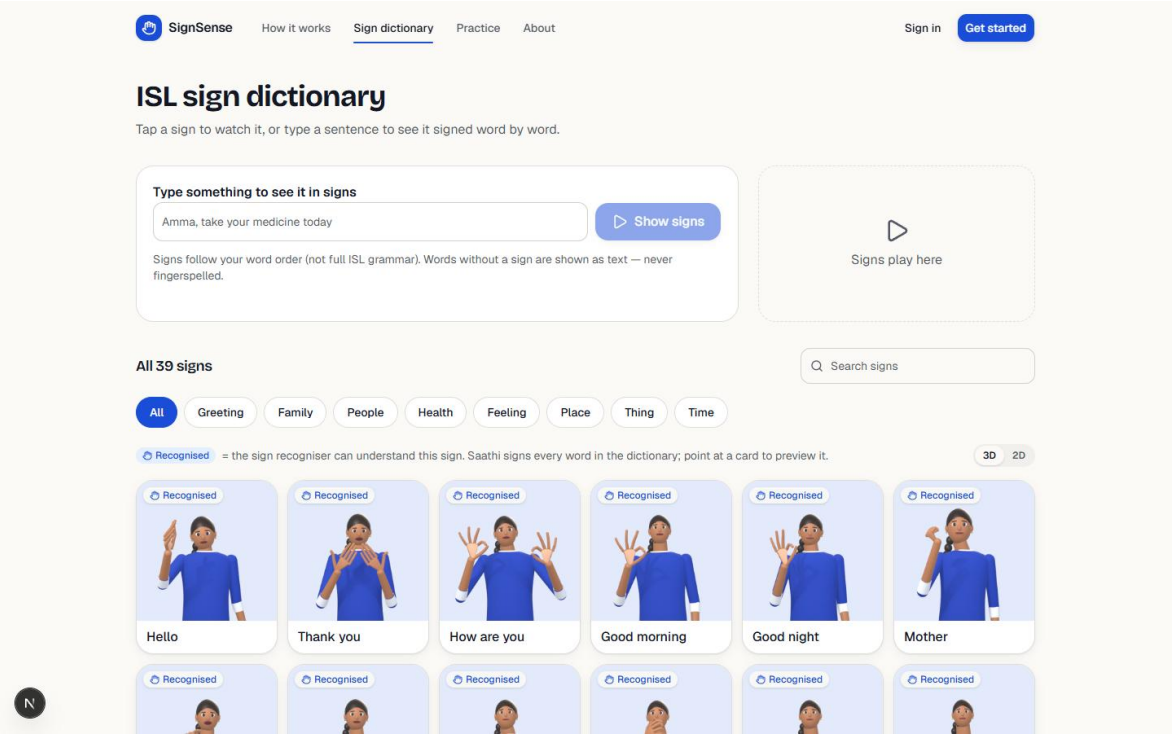
A.2.4 Call Screen



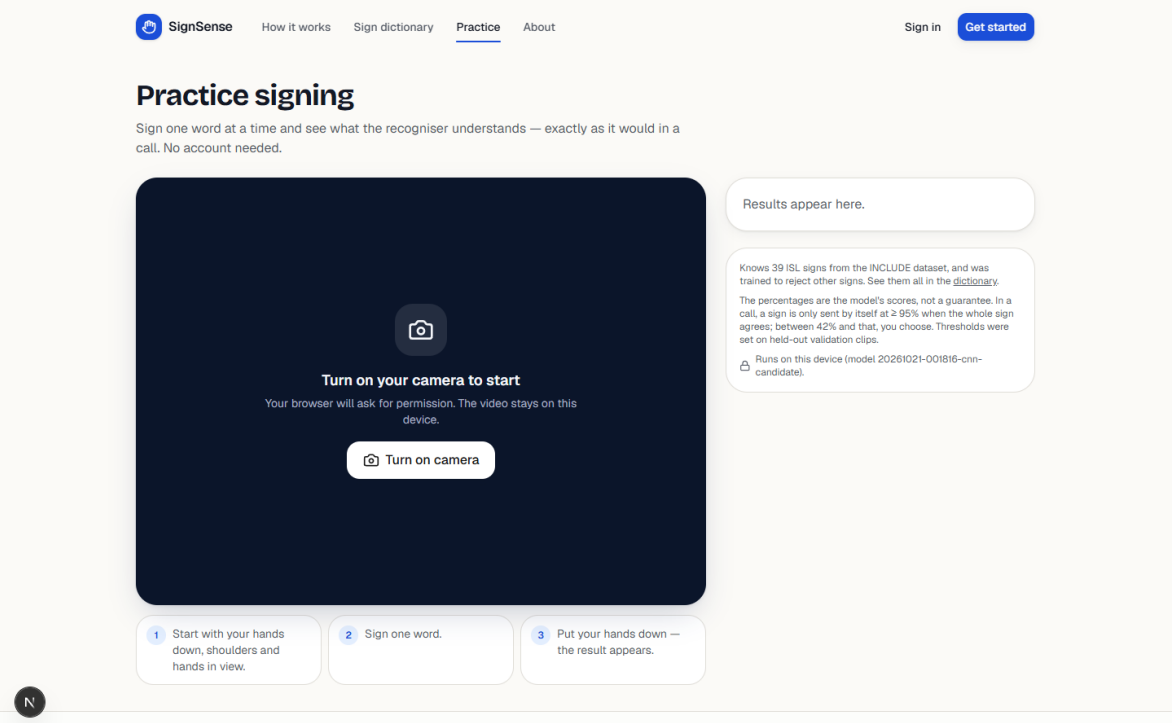
A.2.5 Live Sign Recognition in a Call



A.2.6 Picture Board on a Phone



A.2.7 ISL Sign Dictionary



A.2.8 Practice Page

REFERENCES

1. Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271.
2. Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device real-time body pose tracking. arXiv preprint arXiv:2006.10204.
3. Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. 2016 IEEE European Symposium on Security and Privacy, 292–302.
4. Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., & Bowden, R. (2018). Neural sign language translation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 7784–7793.
5. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. Proceedings of EMNLP 2014, 1724–1734.
6. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. Proceedings of the 34th International Conference on Machine Learning, 1321–1330.
7. Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. International Conference on Learning Representations.
8. Kartynnik, Y., Ablavatski, A., Grishchenko, I., & Grundmann, M. (2019). Real-time facial surface geometry from monocular video on mobile GPUs. arXiv preprint arXiv:1907.06724.
9. Koller, O. (2020). Quantitative survey of the state of the art in sign language recognition. arXiv preprint arXiv:2008.09918.
10. Li, D., Rodriguez, C., Yu, X., & Li, H. (2020). Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 1459–1469.
11. Liu, W., Wang, X., Owens, J., & Li, Y. (2020). Energy-based out-of-distribution detection. Advances in Neural Information Processing Systems, 33, 21464–21475.
12. Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. International Conference on Learning Representations.
13. Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint arXiv:1906.08172.

14. Mahy, R., Matthews, P., & Rosenberg, J. (2010). Traversal Using Relays around NAT (TURN): Relay extensions to Session Traversal Utilities for NAT (STUN) (RFC 5766). Internet Engineering Task Force.
15. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.
16. Rosenberg, J., Mahy, R., Matthews, P., & Wing, D. (2008). Session Traversal Utilities for NAT (STUN) (RFC 5389). Internet Engineering Task Force.
17. Smith, L. N., & Topin, N. (2019). Super-convergence: Very fast training of neural networks using large learning rates. Proceedings of SPIE 11006, Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications.
18. Sridhar, A., Ganesan, R. G., Kumar, P., & Khapra, M. (2020). INCLUDE: A large scale dataset for Indian Sign Language recognition. Proceedings of the 28th ACM International Conference on Multimedia, 1366–1375.
19. W3C. (2021). WebRTC 1.0: Real-time communication between browsers (W3C Recommendation). World Wide Web Consortium.
20. World Health Organization. (2021). World report on hearing. World Health Organization.
21. Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. arXiv preprint arXiv:2006.10214.

PROJECT TITLE	**SIGNSENSE 2.0: ON-DEVICE INDIAN SIGN LANGUAGE RECOGNITION FOR ACCESSIBLE VIDEO CALLING**	
PROGRAM	**B.Tech. ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**	
PROJECT NUMBER	**[PROJECT NO]**	
NAME OF THE STUDENT	**[REGISTER NUMBER]**	**[STUDENT NAME]**
NAME OF THE SUPERVISOR	**N. Pooranam**	
NAME OF THE SDG GOALS MAPPED	**Reduced Inequalities,** **Industry, Innovation, and Infrastructure**	
MENTION THE SDG GOALS NUMBER	**SDG 10, SDG 9**	
NAME OF THE TRL LEVEL	**Component and/or Breadboard Laboratory Validation in a Laboratory Environment**	
MENTION THE TRL LEVEL	**TRL 4**	

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
✓	✓	✓	✓	✓				✓		✓	✓	

Signature of the Supervisor
(Ms. N. Pooranam)

VENUE AND EXPENDITURE STATEMENT FOR THE PROJECT WORK

Laboratory details where the project is carried out	**AI Lab, AI&DS**
Software / Hardware details	Software: Next.js, TypeScript, FastAPI, PostgreSQL, PyTorch, ONNX Runtime Web, MediaPipe, WebRTC, VS Code, Git. Hardware: laptop with NVIDIA RTX 4060 GPU for training; any laptop or phone with a webcam for use.

Details of the Component and Expenditure

S. No	Name of the Component	Qty	Price / Unit in (Rs.)	Amount (Rs.)
1	Open-source software only (no hardware purchased)	–	Nil	Nil

Signature of the student
([STUDENT NAME])

Signature of the Supervisor
(Ms. N. Pooranam)